RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

Rapport de Travaux Pratiques n°3 et °4

RT 3/2

Representation en l'espace d'état et filtrage de Kalman

Encadré par

Mme Rim Amara Boujemaa

Présenté par

Nour Nejia Slim

Zaineb Kchaou

Sonia Mejri

Année Universitaire : 2025/2026

Contents

1 Partie A **2**
A.1 — Conditions d'application du KF classique . . . 2
A.2 — Distribution a posteriori . . . 2
A.3 — Paramètres, entrées et sorties du KF . . . 2
A.4 — Dérivation des équations de prédiction . . . 3
A.5 — Filtre de Kalman Étendu (EKF) . . . 4
A.6 — Optimalité du KF vs EKF . . . 5
A.7 — Applications du Filtre de Kalman . . . 5

2 Partie B **6**
B.1 — Génération de trajectoire . . . 6
B.2 — Implémentation de l'EKF . . . 7
B.3 — Programme principal . . . 9

3 Partie C **10**
C.1 — Simulation unique : trajectoire, EQM, DtoP . . . 10
C.2 — Effet de l'initialisation de P(0|0) . . . 14
C.3 — Effet moyen du bruit — Monte Carlo . . . 18
C.4 — Identification des performances moyennes . . . 20
C.5 — Effet de la variance du bruit . . . 24
C.6 — Effet de l'initialisation de z_est . . . 25
C.7 — Robustesse vis-à-vis de la variance du bruit d'observation . . . 28
C.8 — Autres études de robustesse . . . 30
C.9 — Comparaison 3 stations vs 5 stations . . . 32
C.10 — EKF avec TDOA . . . 36

1 Partie A

A.1)

Le KF classique s'applique aux modèles d'état **linéaires** avec les conditions suivantes :

C1) Modèle d'état linéaire : L'état $x(n)$ s'écrit de façon linéaire en fonction de $x(n-1)$ et des perturbations $u(n)$. Aussi, l'observation $y(n)$ s'écrit de façon linéaire en fonction de $x(n)$ et du bruit $v(n)$.

C2) Modèle d'état gaussien : Les vecteurs perturbations sont supposés gaussiens :

$$u(n) \sim \mathcal{N}(0, Q_U) \qquad \text{et} \qquad v(n) \sim \mathcal{N}(0, Q_v)$$

C3) Perturbations décorrélées : $u(n)$ et $v(n)$ sont décorrélés, c'est-à-dire :

$$E\{u(n)v^T(n)\} = E\{u(n)\}E\{v^T(n)\}$$

A.2)

$p(x(n) \mid y_{0:n})$: C'est la distribution (ou densité de probabilité) a posteriori de l'état $x(n)$, conditionnée par l'ensemble des observations disponibles jusqu'à l'instant n.

- Sous les hypothèses C1 (linéarité), C2 (bruits gaussiens) et C3 (décorrélation), la distribution a posteriori de l'état est une loi Gaussienne (Normale) :

$$x(n) \mid y_{0:n} \sim \mathcal{N}\Big(\hat{x}(n|n),\ \hat{P}(n|n)\Big)$$

Elle est entièrement caractérisée par deux paramètres :

- Sa moyenne : $\hat{x}(n|n)$
- Sa matrice de covariance : $\hat{P}(n|n)$

L'estimation $\hat{x}(n|n)$ représente la meilleure estimation de l'état au sens du minimum de l'erreur quadratique moyenne, obtenue en combinant la prédiction du modèle et la mesure courante via le gain de Kalman. Elle correspond à la moyenne (espérance mathématique) de cette distribution a posteriori.

A.3)

1. Les Paramètres du Filtre :

- $A(n)$: La matrice de transition d'état.
- $H(n)$: La matrice d'observation.
- $Q_{u(n)}$: La matrice de covariance du bruit de processus.
- $Q_{v(n)}$: La matrice de covariance du bruit d'observation.

2. Les Entrées :

À chaque instant n, le filtre a besoin de trois éléments pour fonctionner :

- $y(n)$: L'observation actuelle. C'est la mesure brute bruitée venant du monde réel.

- $\widehat{x}(n-1 \mid n-1)$: L'estimation de l'état précédent.
- $\widehat{P}(n-1 \mid n-1)$: La matrice de covariance précédente.

3. Les Sorties :

Le filtre produit deux résultats principaux qui serviront d'entrées pour l'itération suivante :

- $\widehat{x}(n|n)$: L'**état filtré** (ou mis à jour). C'est l'estimée optimale au sens de l'Erreur Quadratique Moyenne Minimale (EQMM).
- $\widehat{P}(n|n)$: La **matrice de covariance de l'état filtré**. Elle quantifie l'incertitude restante après avoir pris en compte la nouvelle mesure.

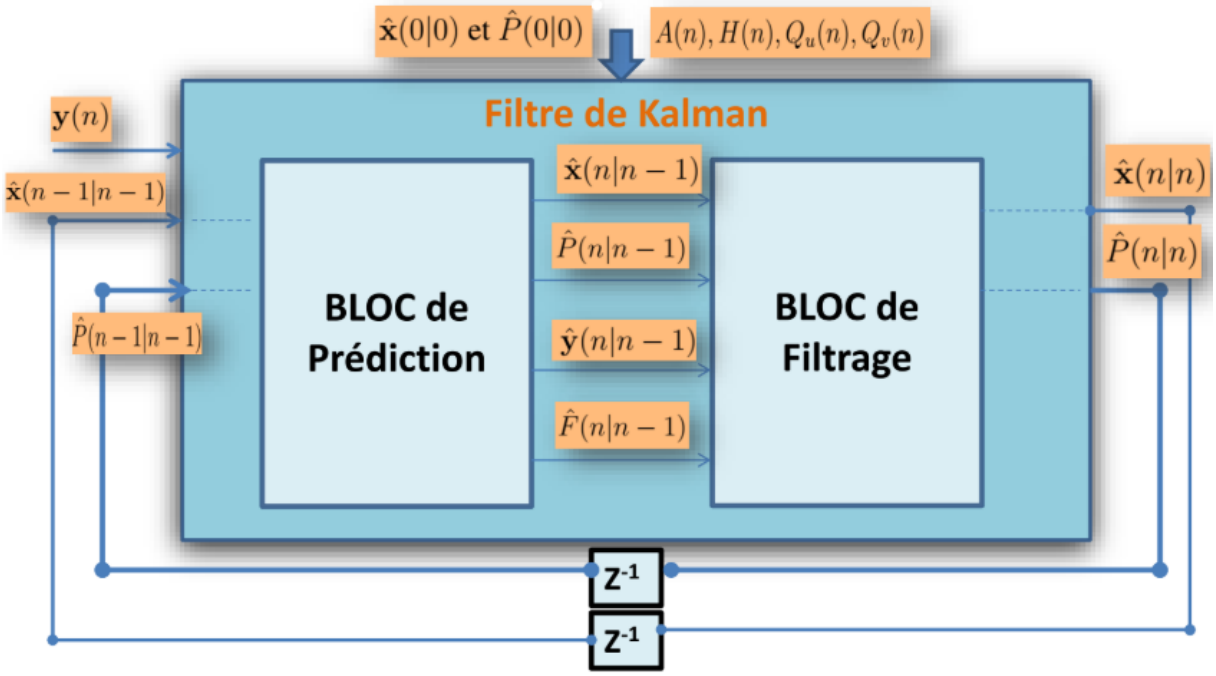


Figure 1: Schéma du filtre de Kalman — paramètres, entrées et sorties

A.4)

État prédit :

On considère le modèle linéaire gaussien : $x(n) = A\,x(n-1) + u(n)$

Par définition, l'estimation a priori (prédite) est l'espérance conditionnelle de l'état sachant toutes les observations jusqu'à $n-1$:

$$\widehat{x}(n \mid n-1) = E[x(n) \mid y_{0:n-1}]$$

$$\widehat{x}(n \mid n-1) = E[A(n)\,x(n-1) + u(n) \mid y_{0:n-1}]$$

Par linéarité de l'espérance conditionnelle :

$$\widehat{x}(n \mid n-1) = A(n)\,E[x(n-1) \mid y_{0:n-1}] + E[u(n) \mid y_{0:n-1}]$$

Or :

- $E[x(n-1) \mid y_{0:n-1}]$ est par définition l'estimation a posteriori de l'instant précédent.
- $E[u(n) \mid y_{0:n-1}] = 0$ car le bruit de processus est blanc et non corrélé avec le passé.

On obtient donc :

$$\widehat{x}(n \mid n-1) = A(n)\,\hat{x}(n-1 \mid n-1)$$

Covariance :

La matrice de covariance de l'erreur de prédiction est définie par :

$$\widehat{P}(n \mid n-1) = E\big[(x(n) - \hat{x}(n|n-1))(x(n) - \hat{x}(n|n-1))^T \mid y_{0:n-1}\big]$$

Définissons l'erreur de prédiction :

$$x(n) - \widehat{x}(n \mid n-1) = A\,(x(n-1) - \widehat{x}(n-1 \mid n-1)) + u(n)$$

Donc :

$$P(n \mid n-1) = E\big[(Ae(n-1) + u(n))(Ae(n-1) + u(n))^T\big]$$
$$\text{avec } e(n-1) = x(n-1) - \widehat{x}(n-1 \mid n-1)$$

En développant :

$$\widehat{P}(n \mid n-1) = A(n)\,\widehat{P}(n-1 \mid n-1)\,A^T(n) + Q_u(n)$$

- **On initialise les valeurs de $\widehat{x}(0|0)$ et $\widehat{P}(0|0)$.**

A.5)

Linéarisation des transformations non linéaires autour de certains points de fonctionnement + application du FK → filtre de Kalman étendu

Lorsque l'équation d'observation est non linéaire :

$$y(n) = h(x(n)) + v(n)$$

On ne peut pas appliquer directement le filtre de Kalman classique (qui exige la condition C1 : linéarité). L'astuce consiste à approximer $h(x(n))$ par une fonction linéaire au voisinage d'un point de fonctionnement bien choisi.

On approxime la fonction non linéaire $h(x(n))$ au voisinage de l'état prédit $\widehat{x}(n \mid n-1)$:

$$h(x(n)) \approx h(\widehat{x}(n \mid n-1)) + H(n) \cdot (x(n) - \widehat{x}(n \mid n-1))$$

où $H(n)$ est la matrice Jacobienne évaluée en $\widehat{x}(n \mid n-1)$.

L'EKF consiste donc à :

1. Linéariser $h(x)$ autour de $\hat{x}(n \mid n-1)$ à chaque pas n
2. Calculer la Jacobienne $H(n)$ en ce point
3. Appliquer les équations classiques du KF au modèle linéarisé
4. Mettre à jour $\hat{x}(n \mid n)$ et $\hat{P}(n \mid n)$ comme dans le KF standard

A.6)

Le KF est optimal uniquement si :

- modèle exact
- bruit gaussien
- modèle linéaire

L'EKF est un estimateur **sous-optimal (approximatif)**. Il ne garantit plus l'optimalité au sens de l'Erreur Quadratique Moyenne Minimale (EQMM) car les hypothèses fondamentales du filtre de Kalman classique ne sont plus strictement vérifiées après linéarisation.

Table 1: Validité des hypothèses après linéarisation (EKF)

Hypothèse	**Validité après linéarisation (EKF)**	**Explication**
C1 : Linéarité	Valide localement & approximativement	La fonction $h(x)$ est remplacée par son développement de Taylor à l'ordre 1 autour de $\hat{x}(n\|n-1)$. Le modèle devient linéaire en $x(n)$, mais uniquement au voisinage du point de linéarisation.
C2 : Gaussianité des bruits	Non vérifiée strictement	Le bruit effectif dans l'équation linéarisée est $v'(n) = v(n) + e(n)$ où $v(n) \sim \mathcal{N}(0, Q_v)$ (gaussien par hypothèse initiale) mais $e(n)$ n'est pas gaussien : il dépend de la non-linéarité de $h(\cdot)$ et de la distribution de l'état estimé. La somme $v'(n)$ n'est donc pas garantie gaussienne.
C3 : Décorrélation / Bruit blanc	Compromise	L'erreur de linéarisation $e(n)$ dépend de $\hat{x}(n\|n-1)$, qui est lui-même calculé à partir de l'historique des mesures $y_{0:n-1}$. Ainsi $e(n)$ (et donc $v'(n)$) peut devenir corrélé avec le passé ou avec l'état, ce qui brise l'hypothèse de bruit blanc indépendant requise par C3.

A.7)

Voici une sélection des applications du Filtre de Kalman classées par domaine, avec des explications techniques :

1. Véhicules Autonomes & Robotique Mobile :

C'est un domaine où le KF/EKF joue un rôle critique pour la sécurité et où la fusion de capteurs est essentielle.

Table 2: Applications du KF en véhicules autonomes

Sous-domaine	**Rôle du Kalman Filter**
Localisation	Fusion GPS/IMU/odométrie pour estimer la position réelle du véhicule
Suivi d'objets	Prédiction de la trajectoire des piétons/véhicules environnants
Navigation AGV	Guidage des robots industriels dans les entrepôts automatisés
SLAM	Cartographie simultanée et localisation pour robots mobiles

2. Navigation Aérospatiale & Systèmes de Guidage :

C'est le domaine historique du KF (projet Apollo) où la précision est vitale.

Applications emblématiques :

- **Programme Apollo (1960s) :** Estimation de trajectoire pour les missions lunaires. Le filtre de Kalman a été théorisé en 1960 par Rudolf Kalman, puis implémenté pour la première fois en 1961 par Stanley Schmidt au sein de la NASA pour la navigation du programme Apollo. Cette application historique a démontré l'efficacité du filtre dans un contexte réel à haute criticité, lançant son adoption massive dans l'aérospatial, la robotique, et bien d'autres domaines.
- **Missiles et Défense :** Guidage du système Patriot.
- **Satellites & Drones :** Navigation autonome avec capteurs limités.

2 Partie B

B.1)

Démarche :

Cette fonction génère la vraie trajectoire du mobile selon le modèle d'état avec bruit d'accélération et calcule les TOA non bruités vers chaque balise.

Implémentation MATLAB :

```
function [Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, S_seed, Anchor_position)
    if ~isempty(S_seed)
        randn('seed', S_seed);
    end

    A = [1 0 Ts 0; 0 1 0 Ts; 0 0 1 0; 0 0 0 1];
    B = [Ts^2/2 0; 0 Ts^2/2; Ts 0;0 Ts];

    Zmat = zeros(4, N);
    Zmat(:, 1) = Zinit;
    c = 3e8; % Vitesse de la lumière

    Ns = size(Anchor_position, 2);
    Yf = zeros(Ns, N);

    for n = 1:N-1
        u = sigmab^2 * randn(2,1); %sigmab c l'amplitude du bruit
        Zmat(:, n+1) = A * Zmat(:, n) + B * u;

        % Génération des TOA non bruités (Modèle d'observation)
        for s = 1:Ns
            dist = norm(Zmat(1:2, n+1) - Anchor_position(:, s));
            Yf(s, n+1) = dist / c;
        end
    end
end
```

Figure 2: Fonction de génération de trajectoire — code MATLAB

Interprétation :

À chaque instant n, un vecteur d'accélération aléatoire $u \sim \mathcal{N}(0, \sigma_b^2 \cdot I_2)$ est généré. Il est propagé via B pour mettre à jour position et vitesse. Les temps d'arrivée non bruités sont calculés en utilisant les positions des stations de base et la distance entre chaque station et la position de la source. Ces temps d'arrivée non bruités sont stockés dans la matrice `Yf`. `S_seed` garantit la reproductibilité des simulations.

B.2)

Démarche :

Cette fonction implémente l'algorithme EKF complet.

Implémentation MATLAB complète :

```
function [Zmat_est, EQM, DtoP, norm_P_est] = EKF_track_traj(Zmat, Ynoisy, zest, Pest, sigmab, sigmav, Ts, N,Anchor_position)
    % Initialisation des matrices et variables de sortie
    A = [1 0 Ts 0; 0 1 0 Ts; 0 0 1 0; 0 0 0 1];
    B = [Ts^2/2 0; 0 Ts^2/2; Ts 0; 0 Ts];
    Q = sigmab^2* (B * B'); % Covariance du bruit de processus
    R = sigmav^2* eye(size(Ynoisy,1)); % Covariance du bruit de mesure
    c_inv = 1 / 3e8;
    Zmat_est = zeros(4, N);
    EQM = zeros(1, N);
    DtoP = zeros(1, N);
    norm_P_est = zeros(1, N);
    curr_z = zest;
    curr_P = Pest;
    cum_err_sq = 0;
    cum_dist = 0;
```

```
function [Zmat_est, EQM, DtoP, norm_P_est] = EKF_track_traj(Zmat, Ynoisy, zest, Pest, sigmau, sigmav, Ts, N,Anchor_position)
    % Initialisation des matrices et variables de sortie
    A = [1 0 Ts 0; 0 1 0 Ts; 0 0 1 0; 0 0 0 1];
    B = [Ts^2/2 0; 0 Ts^2/2; Ts 0; 0 Ts];
    Q = sigmau^2* (B * B'); % Covariance du bruit de processus
    R = sigmav^2* eye(size(Ynoisy,1)); % Covariance du bruit de mesure
    c_inv = 1 / 3e8;

    Zmat_est = zeros(4, N);
    EQM = zeros(1, N);
    DtoP = zeros(1, N);
    norm_P_est = zeros(1, N);

    curr_z = zest;
    curr_P = Pest;
    cum_err_sq = 0;
    cum_dist = 0;
```

```
    for n = 1:N
        % 1. ÉTAPE DE PRÉDICTION
        z_pred = A * curr_z;
        P_pred = A * curr_P * A' + Q;

        % 2. LINÉARISATION (Jacobienne H)
        Ns = size(Ynoisy, 1);
        H = zeros(Ns, 4);
        y_pred = zeros(Ns, 1);
        for s = 1:Ns
            diff = z_pred(1:2) - Anchor_position(:, s); % Nécessite Anchor_position en global ou argument
            d = norm(diff);
            y_pred(s) = d * c_inv;
            H(s, 1:2) = c_inv * (diff' / d); % Dérivée de la distance
        end

        % 3. ÉTAPE DE MISE À JOUR (Correction)
        F = H * P_pred * H' + R; % Covariance de l'innovation
        K = P_pred * H' / F;    % Gain de Kalman

        curr_z = z_pred + K * (Ynoisy(:, n) - y_pred);
        curr_P = P_pred - K * H * P_pred;

        % Stockage et calcul des performances
        Zmat_est(:, n) = curr_z;
        dist_err = norm(Zmat(1:2, n) - curr_z(1:2));
        cum_dist = cum_dist + dist_err;
        cum_err_sq = cum_err_sq + dist_err^2;

        DtoP(n) = cum_dist / n;
        EQM(n) = cum_err_sq / n;
        norm_P_est(n) = trace(curr_P);
    end
end
```

Interprétation :

Cette fonction est une implémentation d'un filtre de Kalman étendu (EKF) pour la poursuite d'une trajectoire en utilisant des mesures de temps d'arrivée (TOA) provenant de

plusieurs stations de base fixes. L'objectif est de déterminer la trajectoire suivie par un mobile à partir des temps d'arrivée mesurés avec du bruit.

Le filtre EKF suit un processus de mise à jour et de prédiction pour estimer la trajectoire à partir des mesures TOA. Le filtre prédit d'abord la position de l'objet à chaque étape de temps en utilisant l'équation de prédiction standard. Ensuite, il utilise la matrice de covariance de l'estimation pour calculer la matrice H, qui est la dérivée de la fonction de mesure. La matrice H est ensuite utilisée pour calculer la prédiction des observations et la matrice de covariance de l'erreur de prédiction.

Ensuite, le filtre EKF effectue une étape de mise à jour en utilisant la matrice de gain de Kalman pour calculer l'estimation de la trajectoire à partir de la prédiction de la position et des mesures TOA. La matrice de covariance de l'estimation est également mise à jour en fonction de l'estimation de la trajectoire. Les matrices `EQM`, `DtoP` et `norm_P_est` sont utilisées pour évaluer la qualité de l'estimation.

Le principal avantage du filtre EKF est qu'il permet une estimation précise de la trajectoire de l'objet en présence de bruit dans les mesures. Cependant, l'utilisation d'un modèle de processus linéaire peut être une limitation si le système suit un comportement non linéaire complexe.

B.3)

Démarche et Implémentation :

Ce script orchestre la génération de trajectoire, le bruitage et l'appel à l'EKF.

```
    for n = 1:N
        % 1. ÉTAPE DE PRÉDICTION
        z_pred = A * curr_z;
        P_pred = A * curr_P * A' + Q;

        % 2. LINÉARISATION (Jacobienne H)
        Ns = size(Ynoisy, 1);
        H = zeros(Ns, 4);
        y_pred = zeros(Ns, 1);
        for s = 1:Ns
            diff = z_pred(1:2) - Anchor_position(:, s); % Nécessite Anchor_position en global ou argument
            d = norm(diff);
            y_pred(s) = d * c_inv;
            H(s, 1:2) = c_inv * (diff' / d); % Dérivée de la distance
        end

        % 3. ÉTAPE DE MISE À JOUR (Correction)
        F = H * P_pred * H' + R; % Covariance de l'innovation
        K = P_pred * H' / F;    % Gain de Kalman

        curr_z = z_pred + K * (Ynoisy(:, n) - y_pred);
        curr_P = P_pred - K * H * P_pred;

        % Stockage et calcul des performances
        Zmat_est(:, n) = curr_z;
        dist_err = norm(Zmat(1:2, n) - curr_z(1:2));
        cum_dist = cum_dist + dist_err;
        cum_err_sq = cum_err_sq + dist_err^2;

        DtoP(n) = cum_dist / n;
        EQM(n) = cum_err_sq / n;
        norm_P_est(n) = trace(curr_P);
    end
end
```

Figure 3: Programme principal — affichage des résultats

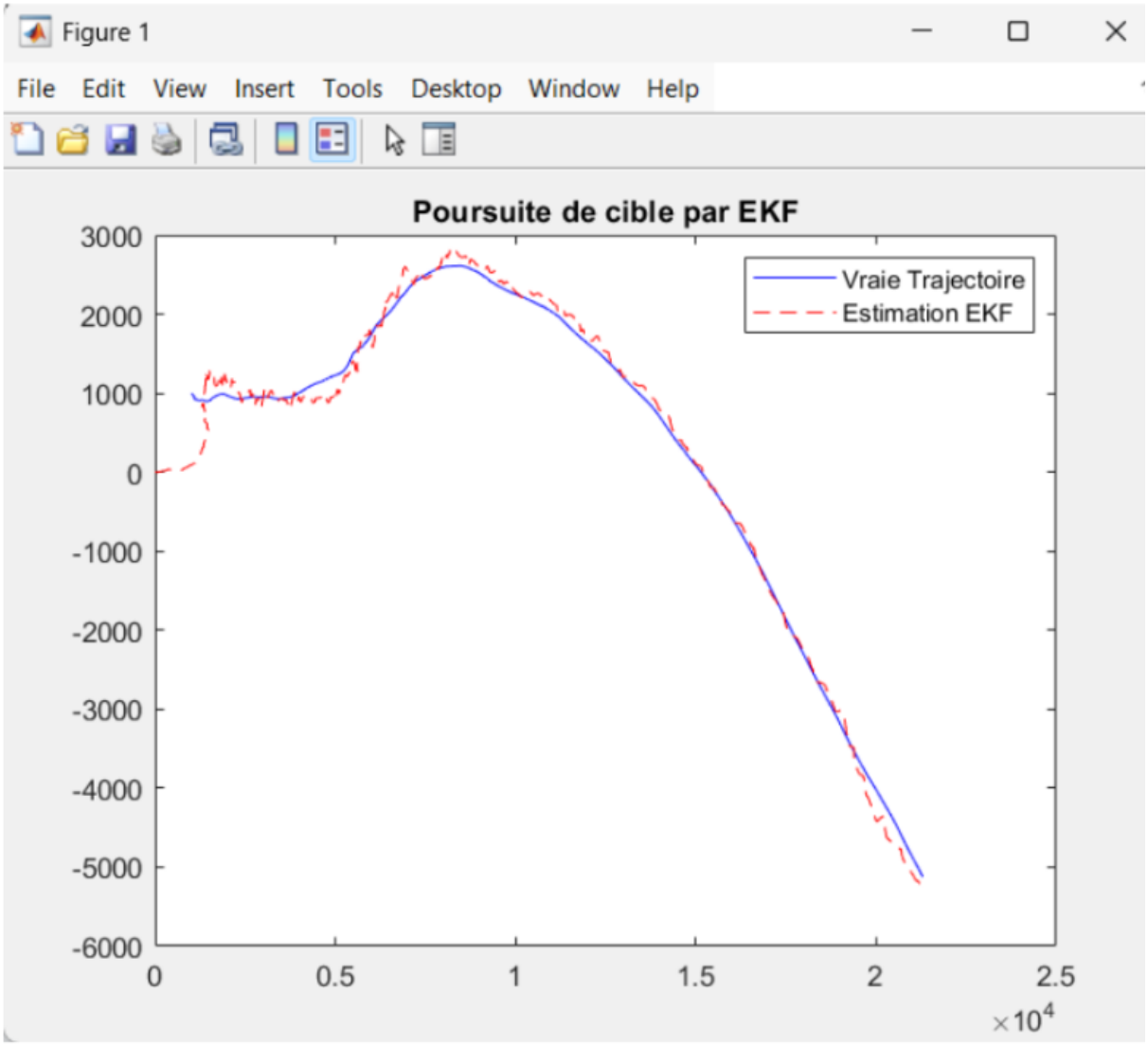
3 Partie C

C.1

Pour $\sigma_v = 10^{-12}$ et $N_w = 1$ (une seule simulation), afficher la vraie trajectoire ainsi que celle obtenue par EKF (figure 1), l'EQM (figure 2), le DtoP et norm_P_est sur la même figure 3. Indiquer quelle performance est étudiée par ces différentes mesures et interpréter les courbes.

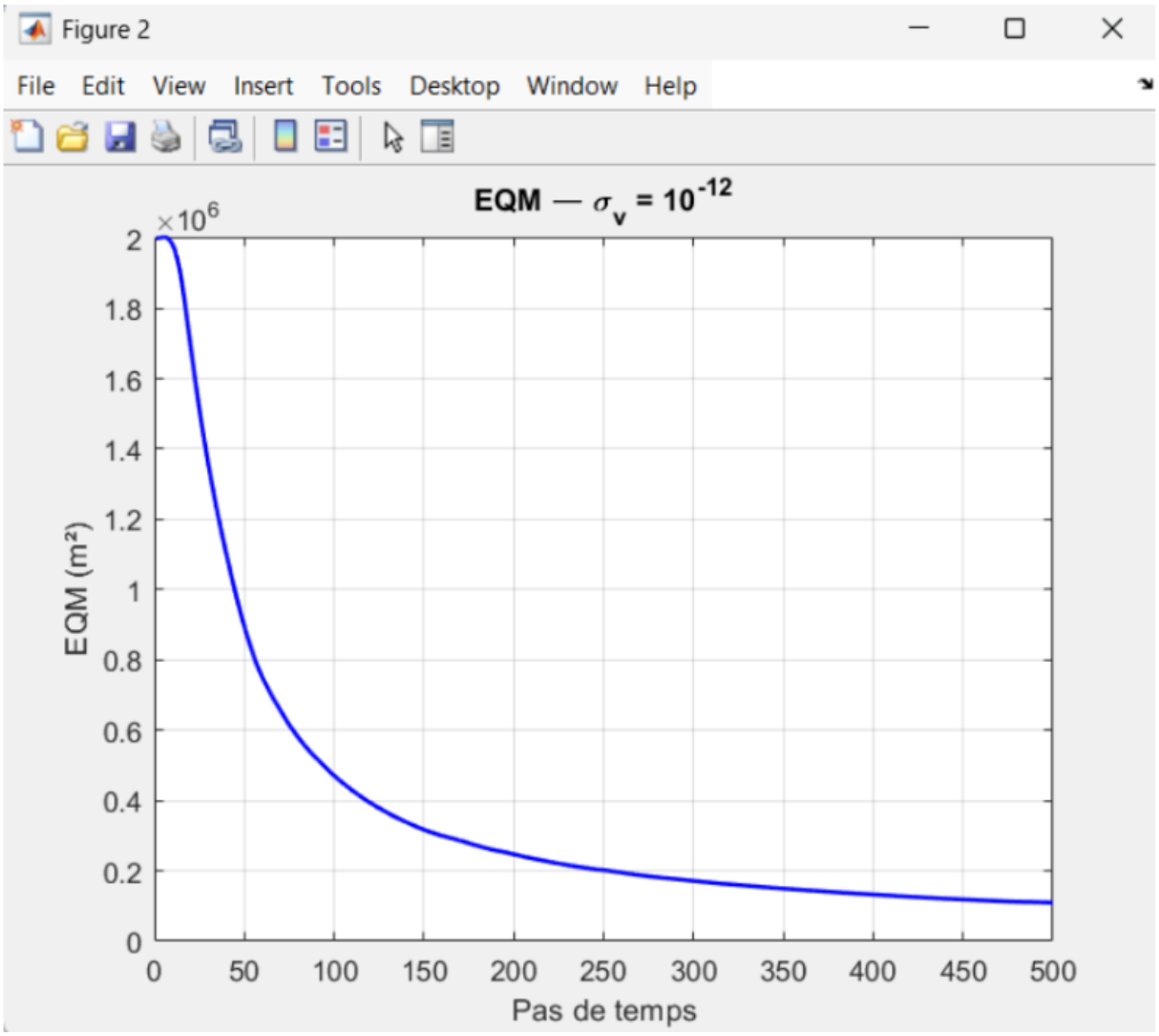
```
N = 500;
NS=3;
sigmav =sqrt(1e-12);
sigmab=sqrt(2);
N_w = 1;
EQM_mat = zeros(N_w, N);
DtoP_mat = zeros(N_w, N);
norm_P_est_mat = zeros(N_w, N);
C = 3 * 10^8;
z_est_init = [0; 0; 0; 0];
P_est_init = 0.01 * eye(4);
Ts = 1;
P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];
Anchor_pos = [P1 P2 P3];
Zinit = [1000; 1000; 0; 0];
S_seed = 50;
%Génération de la trajectoire réelle et des TOA non bruités
[Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, S_seed, Anchor_pos);
for nw=1:N_w
    Ynoisy = Yf + sigmav * randn(NS,N);
    [Zmat_est, EQM, DtoP, norm_P_est] = EKF_track_traj(Zmat, Ynoisy, z_est_init, P_est_init, sigmab, sigmav, Ts, N,Anchor_pos);
    EQM_mat(nw, :) = EQM;
    DtoP_mat(nw, :) = DtoP;
    norm_P_est_mat(nw, :) = norm_P_est;
end
```

Figure 1 :



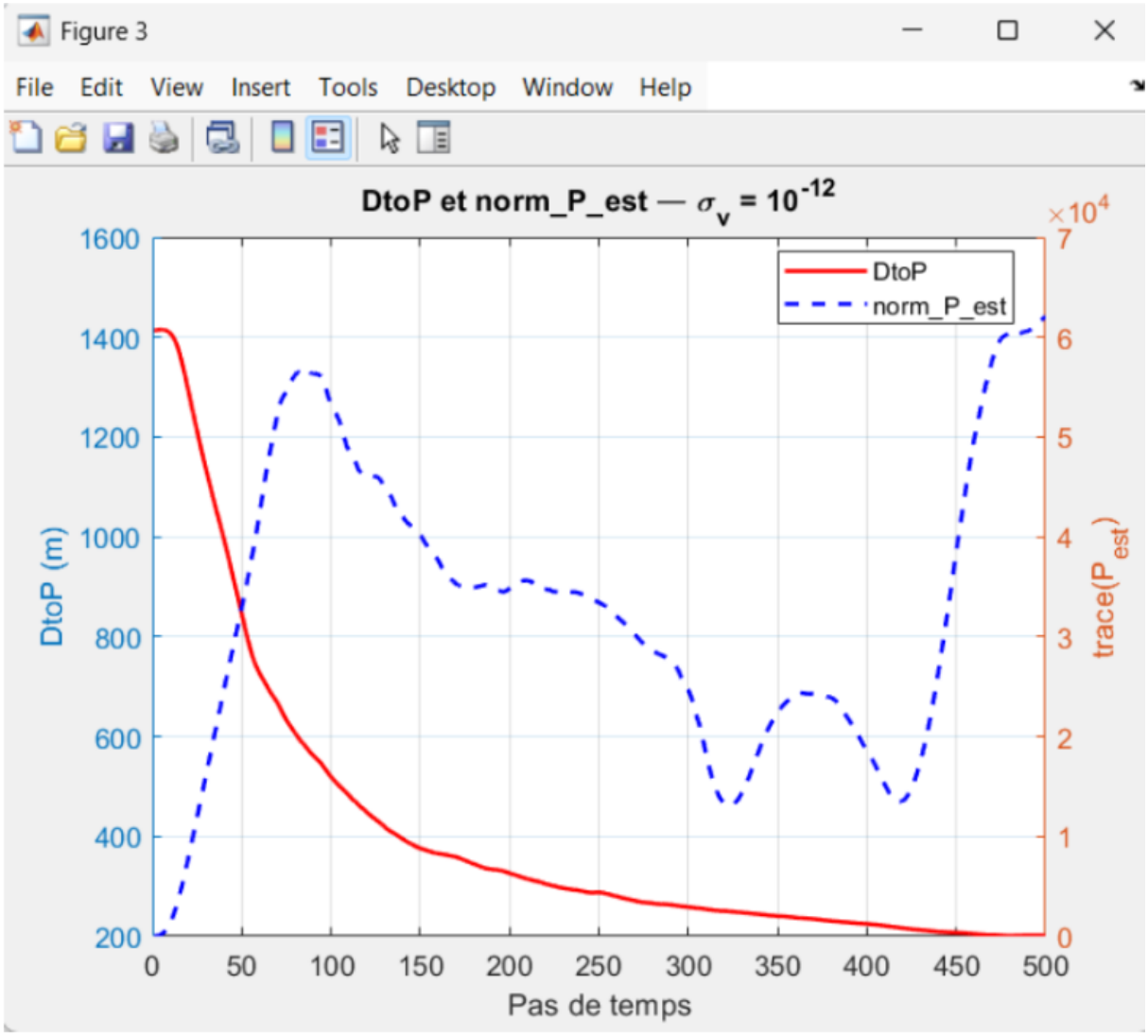
D'après la figure, on peut constater que malgré une présence d'erreur, l'EKF converge rapidement vers la vraie trajectoire. Après une brève phase transitoire, les deux courbes se superposent quasi parfaitement.

Figure 2 :



L'EQM mesure la précision de l'estimation EKF. Elle décroît rapidement au fil du temps, indiquant une convergence du filtre, puis se stabilise à un niveau très bas ($\sim$ quelques m^2), confirmant que l'estimation devient de plus en plus précise au fur et à mesure des itérations.

Figure 3 :



Interprétation des courbes (DtoP et norm_P_est) :

Courbe rouge : DtoP (Distance to True Position)

La courbe DtoP représente l'erreur réelle de position entre l'estimation fournie par l'EKF et la position vraie du mobile.

Au début de la simulation ($n \approx 0$ à 50), l'erreur est très élevée ($\sim$ 1400 m), ce qui est normal car l'état initial du filtre est très éloigné de la position réelle. Le filtre exploite alors fortement les premières mesures pour corriger cette erreur, ce qui explique la chute rapide de la courbe.

Ensuite ($n \approx 50$ à 200), la décroissance devient plus progressive. Le filtre entre dans une phase d'ajustement plus fine où il affine son estimation tout en suivant la dynamique du système.

À partir de $n \approx 200$, la courbe continue de diminuer lentement et tend vers une valeur faible ($\sim$ 200 m). Cela indique que le filtre se rapproche du régime de convergence, même si une erreur résiduelle subsiste. Cette erreur est due aux bruits présents dans le système et aux approximations liées à la linéarisation de l'EKF.

Courbe bleue : norm_P_est = Tr($\hat{P}(n|n)$)

La trace de la matrice de covariance représente l'incertitude estimée par le filtre sur son état.

Au début ($n \approx 0$ à 80), la trace augmente fortement. Cela s'explique par le fait que le filtre part d'une mauvaise estimation initiale et que l'étape de prédiction amplifie cette incertitude, notamment à cause du bruit de processus.

Ensuite ($n \approx 80$ à 300), la trace diminue globalement, ce qui traduit une amélioration de la confiance du filtre dans son estimation grâce aux corrections apportées par les mesures.

Cependant, on observe ensuite des fluctuations importantes ($n \approx 300$ à 450), suivies d'une forte remontée en fin de simulation. Ce comportement indique que l'incertitude estimée par le filtre n'est pas stable. Cela peut être dû à la nature non linéaire du modèle (EKF), à des erreurs de linéarisation, ou à un déséquilibre entre les matrices de bruit (Q et R).

Interprétation globale :

- Le **DtoP** mesure la performance réelle du filtre (erreur de position).
- La **trace de** $\hat{P}$ mesure la confiance du filtre dans son estimation.

On observe que même si l'erreur réelle diminue de manière régulière, l'incertitude estimée par le filtre n'est pas parfaitement cohérente, ce qui est un comportement classique du filtre de Kalman étendu dans des systèmes non linéaires.

Ces résultats montrent que l'EKF améliore progressivement l'estimation de la position, mais que sa confiance interne ($\hat{P}$) peut présenter des variations importantes, notamment en raison des approximations de linéarisation.

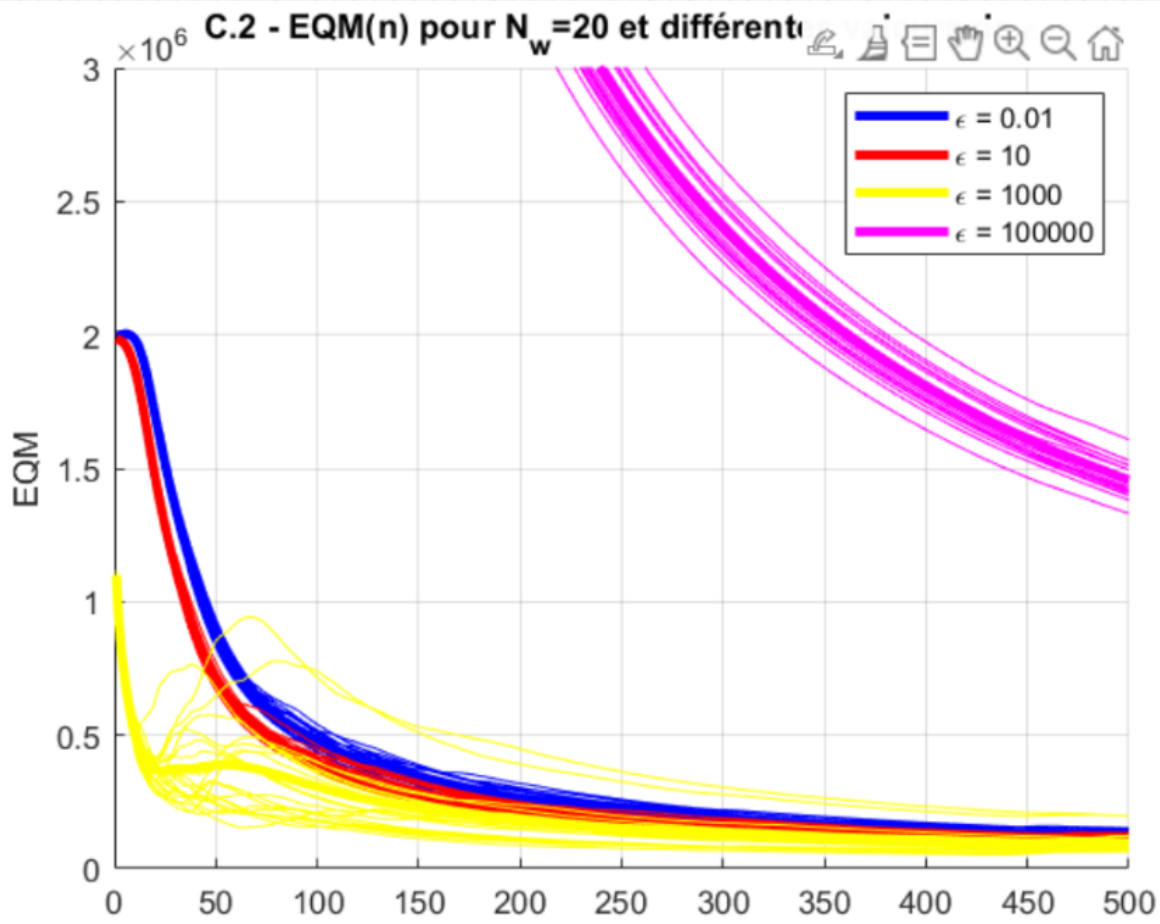
C.2

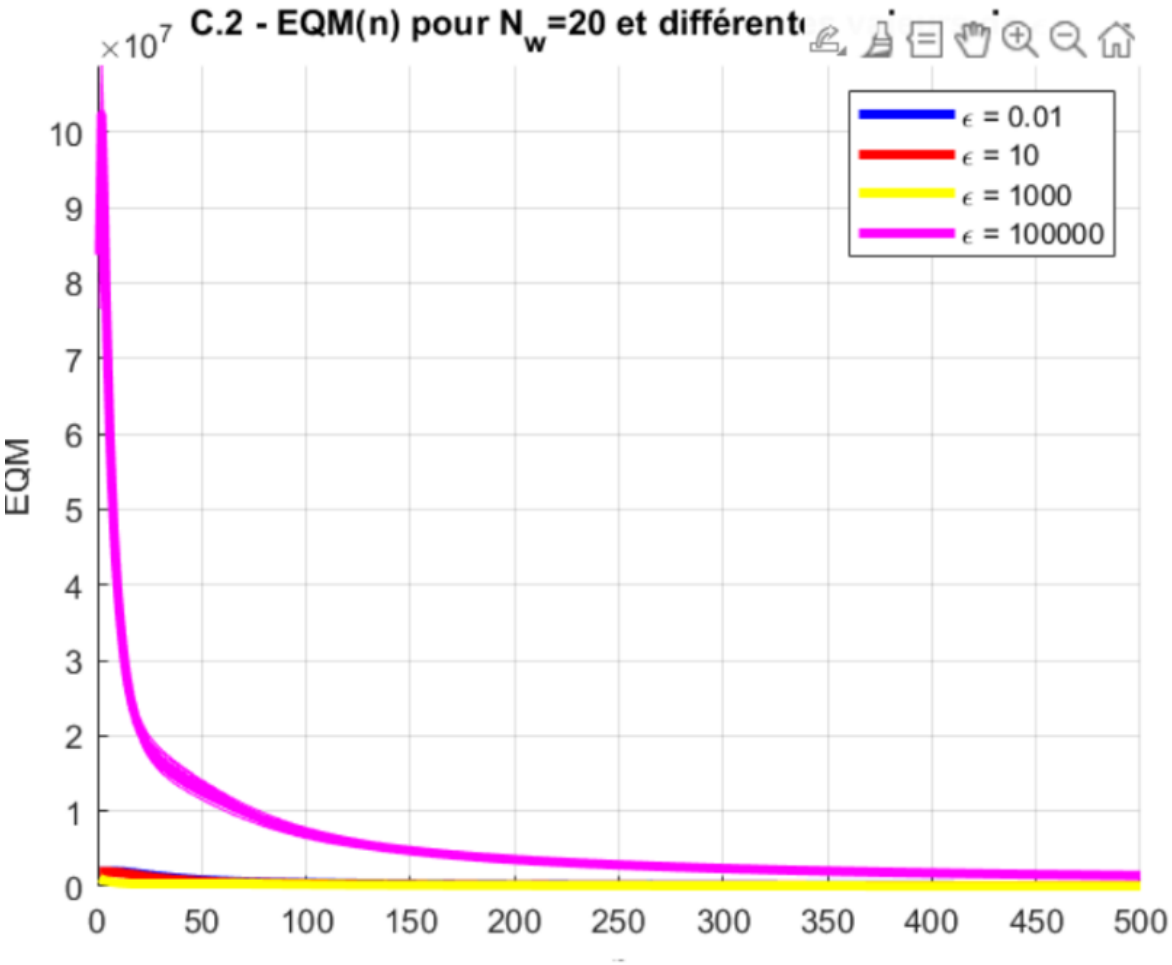
Pour $\varepsilon = 0.01,\ 10,\ 10^3,\ 10^5$ afficher les EQM obtenues pour $N_w = 20$ réalisations de l'EKF sur un même graphe. Interpréter quant à l'effet de l'initialisation de la matrice de covariance $\hat{P}(0|0)$.

Code :

```
N = 500;Nw = 20;NS=3;
epsilon_values = [0.01, 10, 1000, 1e5];
sigmav =sqrt(1e-12);sigmab=sqrt(2);
P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];
Anchor_pos = [P1 P2 P3]; Zinit = [1000; 1000; 0; 0];
colors = {'b', 'r', 'y', 'm'};
EQM_mat = zeros(Nw, N, length(epsilon_values));
[Zmat, Yf] = generate_traj(Zinit, sigmab, 1, N, 50, Anchor_pos);
figure('Color','w'); hold on;
legend_handles = [];
for e = 1:length(epsilon_values)
    epsilon = epsilon_values(e);
    P_est_init = epsilon * eye(4);
    for nw = 1:Nw
        Ynoisy = Yf + sigmav*randn(NS,N);
        [~, EQM, ~, ~] = EKF_track_traj(Zmat, Ynoisy, [0;0;0;0], P_est_init, sigmab, sigmav, 1, N, Anchor_pos);
        EQM_mat(nw,:,e) = EQM;
        %  courbes fines (réalisations)
        plot(1:N, EQM, 'Color', colors{e}, 'LineWidth', 0.5);
    end
    %  moyenne (courbe épaisse)
    EQM_mean = mean(EQM_mat(:,:,e), 1);
    h = plot(1:N, EQM_mean, 'Color', colors{e}, 'LineWidth', 3);
    legend_handles = [legend_handles h];
end
grid on;
xlabel('n');
ylabel('EQM');
title('C.2 - EQM(n) pour N_w=20 et différentes valeurs de \epsilon');
legend(legend_handles, ...
    {'\epsilon = 0.01','\epsilon = 10','\epsilon = 1000','\epsilon = 100000'}, ...
    'Location','northeast');
axis([0 500 0 3e6]);
hold off;
```

Résultats :


C.2 - EQM(n) pour N_w=20 et différent
×10^6
EQM
3
2.5
2
1.5
1
0.5
0
0
50
100
150
200
250
300
350
400
450
500
ϵ = 0.01
ϵ = 10
ϵ = 1000
ϵ = 100000



Interprétation de l'effet de l'initialisation de $P(0|0) = \varepsilon I$:

Les résultats obtenus mettent en évidence une forte dépendance des performances de l'algorithme EKF vis-à-vis du choix de la covariance initiale $P(0|0)$, paramétrée ici par le scalaire ε.

La matrice $P(0|0)$ représente le niveau d'incertitude initiale sur l'état estimé. Ainsi, le paramètre ε contrôle directement la confiance accordée à l'estimation initiale par rapport aux observations.

Lorsque ε est très faible (par exemple $\varepsilon = 0.01$), la covariance initiale est faible, ce qui signifie que le filtre considère l'état initial comme étant très fiable. Dans ce cas, le gain de Kalman est faible lors des premières itérations, ce qui limite l'influence des mesures. Si l'estimation initiale est éloignée de l'état réel, le filtre corrige lentement son erreur, ce qui conduit à une valeur relativement élevée de l'EQM durant la phase transitoire.

À l'inverse, lorsque ε prend des valeurs modérées (par exemple $\varepsilon = 10^3$), le filtre adopte un compromis plus équilibré entre la confiance dans le modèle et dans les observations. Le gain de Kalman est alors suffisamment élevé pour permettre des corrections efficaces sans amplifier excessivement le bruit de mesure. Dans ce cas, l'EQM est minimisée, ce qui correspond à un fonctionnement quasi optimal du filtre.

En revanche, pour des valeurs très élevées de ε (par exemple $\varepsilon = 10^5$), l'incertitude initiale est très grande. Le filtre accorde alors une confiance excessive aux observations, ce qui se

traduit par un gain de Kalman élevé. Cette situation peut engendrer une forte sensibilité au bruit de mesure ainsi qu'aux erreurs de linéarisation inhérentes à l'EKF. Il en résulte une instabilité transitoire et une augmentation significative de l'EQM, notamment au début de la convergence.

Ces observations mettent en évidence l'existence d'un compromis dans le choix de ε. En effet, une valeur trop faible conduit à un filtre rigide, tandis qu'une valeur trop élevée entraîne un comportement instable. Il existe donc une zone optimale de ε permettant de minimiser l'erreur quadratique moyenne tout en assurant une convergence rapide.

Ce comportement est conforme aux résultats classiques de la littérature sur l'EKF, qui soulignent que ses performances dépendent fortement du choix des paramètres de covariance initiaux et que, en raison de la linéarisation locale du modèle non linéaire, une mauvaise initialisation peut conduire à une dégradation significative des performances, voire à une divergence du filtre.

En conclusion, le paramètre ε constitue un élément critique de réglage de l'EKF. Une valeur intermédiaire permet d'obtenir un compromis optimal entre robustesse, rapidité de convergence et précision de l'estimation.

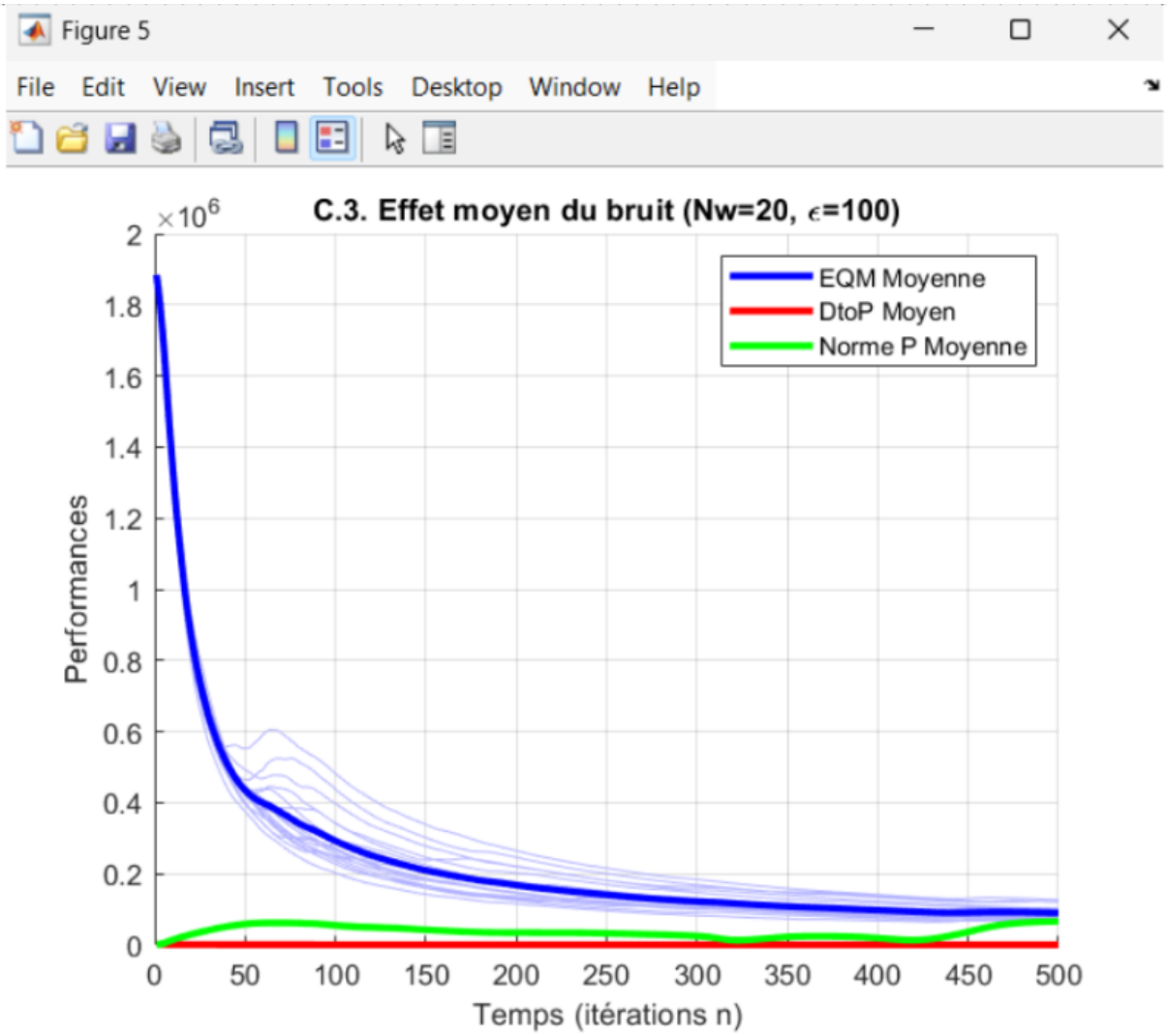
C.3

Effet moyen du bruit sur les TOA pour $\varepsilon = 100$, $N_w = 20$ simulations. Afficher les N_w courbes pour l'EQM (bleu clair) ainsi que leur moyenne (bleu foncé). Faire la même chose pour DtoP (rouge) et norm_P_est (vert). Interpréter quant à la variabilité des résultats.

Code :

```
N = 500;Nw = 20;NS=3;
epsilon = 100;sigmav =sqrt(1e-12);sigmab=sqrt(2);
Ts = 1;
P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];Anchor_pos = [P1 P2 P3]; Zinit = [1000; 1000; 0; 0];
Zinit = [1000; 1000; 0; 0];
z_est_init = [0; 0; 0; 0];
P_est_init = epsilon * eye(4);
EQM_mat = zeros(Nw, N);
DtoP_mat = zeros(Nw, N);
norm_P_est_mat = zeros(Nw, N);
[Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, 50, Anchor_pos);
for nw = 1:Nw
    Ynoisy = Yf + sigmav* randn(NS,N);
    [~, EQM, DtoP, norm_P_est] = EKF_track_traj(Zmat, Ynoisy, z_est_init, P_est_init, sigmab, sigmav, Ts, N, Anchor_pos);
    EQM_mat(nw, :) = EQM;
    DtoP_mat(nw, :) = DtoP;
    norm_P_est_mat(nw, :) = norm_P_est;
end
% 5. Affichage des résultats
figure('Color', 'w'); hold on;
% Affichage des Nw courbes individuelles (couleurs claires)
plot(EQM_mat', 'Color', [0.7, 0.7, 1], 'HandleVisibility', 'off'); % Bleu clair
plot(DtoP_mat', 'Color', [1, 0.7, 0.7], 'HandleVisibility', 'off'); % Rouge clair
plot(norm_P_est_mat', 'Color', [0.7, 1, 0.7], 'HandleVisibility', 'off'); % Vert clair
% Affichage des moyennes (couleurs foncées)
plot(mean(EQM_mat), 'b', 'LineWidth', 2.5, 'DisplayName', 'EQM Moyenne');
plot(mean(DtoP_mat), 'r', 'LineWidth', 2.5, 'DisplayName', 'DtoP Moyen');
plot(mean(norm_P_est_mat), 'g', 'LineWidth', 2.5, 'DisplayName', 'Norme P Moyenne');
% Mise en forme
xlabel('Temps (itérations n)');
ylabel('Performances');
title(['C.3. Effet moyen du bruit (Nw=20, \epsilon=' num2str(epsilon) ')']);
legend('show');
grid on;
```

On effectue 20 réalisations indépendantes (différents tirages du bruit `Y_noisy`) pour étudier la variabilité statistique des performances.



Selon les résultats obtenus, la courbe de l'EQM décroît progressivement jusqu'à converger vers zéro, ce qui traduit une réduction continue de l'erreur entre la trajectoire réelle et celle estimée. Cela confirme la bonne performance de l'Estimateur à Filtre de Kalman Étendu (EKF), capable de fournir une estimation précise au fil du temps.

Par ailleurs, les courbes du DtoP ainsi que la trace (ou norme) de la matrice de covariance estimée restent nulles et constantes. Ce comportement indique que la matrice de covariance est correctement initialisée et que son évolution est déterministe, car elle dépend uniquement de la structure du modèle et non des observations bruitées. Ainsi, toutes les réalisations associées à cette grandeur se superposent.

De plus, la faible variabilité observée entre les différentes simulations, notamment après la phase de convergence, montre que les trajectoires individuelles restent très proches de la moyenne. Cela met en évidence la robustesse de l'EKF vis-à-vis du bruit de mesure.

Enfin, cette faible dispersion permet de conclure que le nombre de simulations Monte Carlo utilisé (20) est suffisant pour obtenir une estimation stable et fiable des performances moyennes de l'algorithme.

C.4

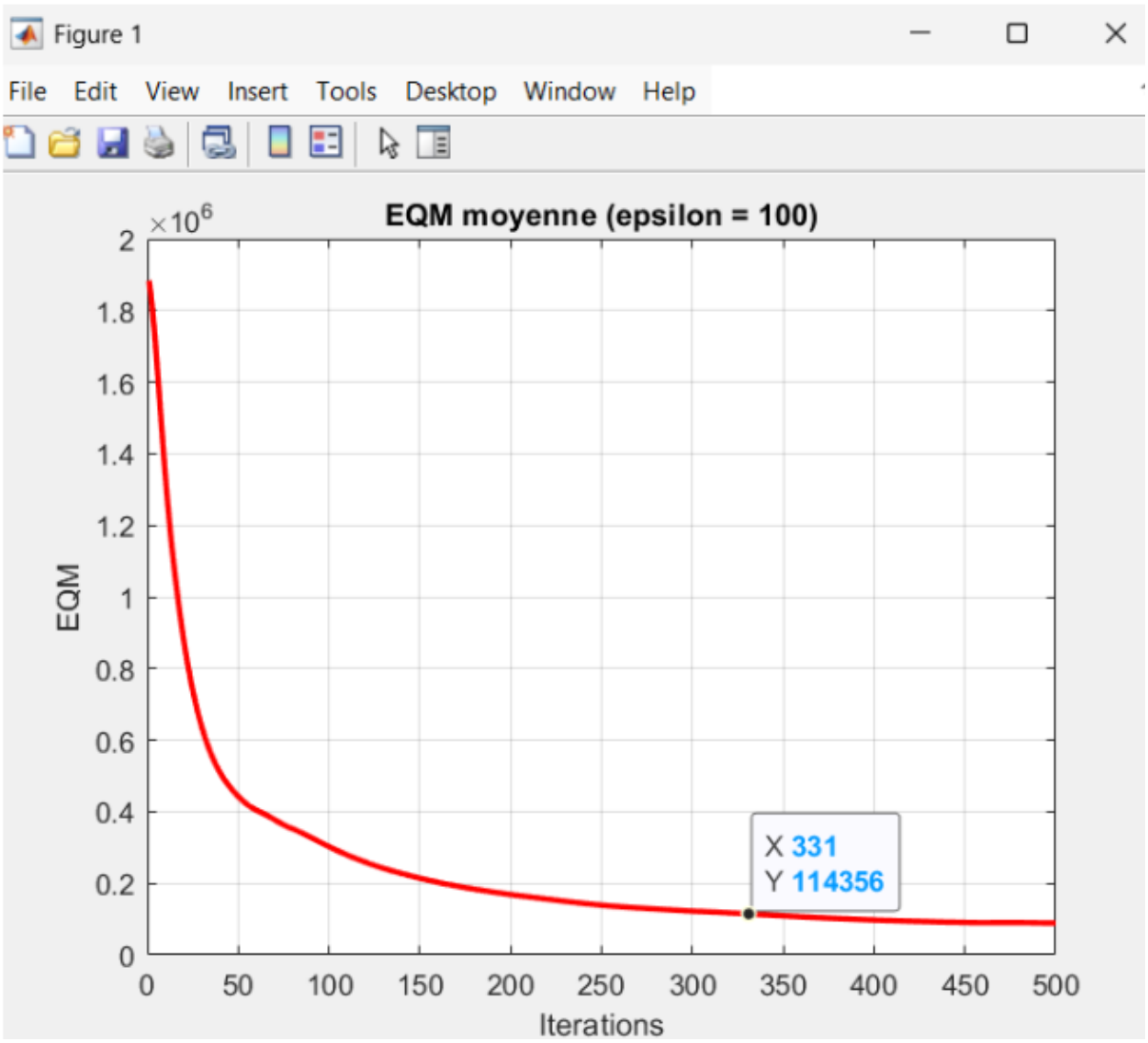
Identifier les performances moyennes. Pour chaque courbe moyenne obtenue : EQM_moy, DtoP_moy, déterminer la performance finale après convergence. Présenter un tableau pour plusieurs valeurs d'eps. Choisir une valeur optimale.

Visuellement et juste avec la courbe :

Code :

```
clc; clear; close all;
N = 500;
Nw = 20;
Ns = 3;
sigmav = sqrt(1e-12);
sigmab = sqrt(2);
P1 = [-1500;  1500];
P2 = [10000;  3000];
P3 = [15000; -1000];
Anchor_pos = [P1 P2 P3];
Zinit = [1000; 1000; 0; 0];
Ts = 1;
epsilon = 100;
zest = [0;0;0;0];
Pest = epsilon * eye(4);
EQM_sum = zeros(1, N);
for w = 1:Nw
    [Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, [], Anchor_pos);
    Ynoisy = Yf + sigmav * randn(Ns, N);
    [~, EQM, ~, ~] = EKF_track_traj( ...
        Zmat, Ynoisy, zest, Pest, sigmab, sigmav, Ts, N, Anchor_pos);
    EQM_sum = EQM_sum + EQM;

end
EQM_moy = EQM_sum / Nw;
figure;
plot(1:N, EQM_moy, 'r', 'LineWidth', 2);
title('EQM moyenne (epsilon = 100)');
xlabel('Iterations');
ylabel('EQM');
grid on;
```



On voit que $n_c = 331$ est une valeur optimale.

Méthode avec epsilon (seuil de tolérance) et eps optimal :

ICI eps est un seuil pour accepter que le filtre converge. Ce n'est PAS l'epsilon d'initialisation de matrice P de covariance !

Ici on a choisi epsilon de l'initialisation = 100.

Code :

```
clc; clear; close all;
N = 500;Nw = 20;Ns = 3;
sigmav = sqrt(1e-12);sigmab = sqrt(2);
P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];Anchor_pos = [P1 P2 P3];
Zinit = [1000; 1000; 0; 0];
Ts = 1;S_seed = 50;epsilon = 100;   zest = [0;0;0;0];
Pest = epsilon * eye(4);
eps_values = [80000,20000, 1000, 500,300,100,50,10,1,0.01,0.00001];
results = zeros(length(eps_values),4);
for i = 1:length(eps_values)
    eps_val = eps_values(i);
    EQM_sum = zeros(1,N);
    DtoP_sum = zeros(1,N);
    for w = 1:Nw
        [Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, [], Anchor_pos);
        Ynoisy = Yf + sigmav * randn(Ns, N);
        [~, EQM, DtoP, ~] = EKF_track_traj( ...
            Zmat, Ynoisy, zest, Pest, sigmab, sigmav, Ts, N, Anchor_pos);
        EQM_sum = EQM_sum + EQM;
        DtoP_sum = DtoP_sum + DtoP;
    end
    EQM_moy = EQM_sum / Nw;
    DtoP_moy = DtoP_sum / Nw;
    nc = N; % valeur par défaut = pas de convergence
    for k = 2:N
        if abs(EQM_moy(k) - EQM_moy(k-1)) < eps_val
            nc = k;
            break;
        end
    end
    EQM_final = mean(EQM_moy(nc:N));
    DtoP_final = mean(DtoP_moy(nc:N));
    results(i,:) = [eps_val, nc, EQM_final, DtoP_final];
end
```

Résultat :

```
---------RESULTATS ------------
        eps          nc        EQM_final       DtoP_final
    8.0e+04           2    325408.907144       390.290452
    2.0e+04          29    182704.217131       314.449948
    1.0e+03         183    125318.698431       250.682472
    5.0e+02         231    108027.221034       230.651748
    3.0e+02         310     97316.600119       214.212412
    1.0e+02         500     93800.249109       215.520699
    5.0e+01         500     81022.087076       199.696737
    1.0e+01         500     79565.800029       198.323728
    1.0e+00         500     86084.296054       207.776096
    1.0e-02         500     89671.412234       211.594523
    1.0e-05         500    103409.910888       223.035650
```

On choisit eps = 300 et $n_c = 310$.

JUSTIFICATION : Le choix du paramètre ε a été effectué en analysant le compromis entre la précision finale et le temps de convergence. Pour des valeurs élevées de ε, la convergence est très rapide mais l'erreur reste importante, traduisant une convergence prématurée. À l'inverse, pour des valeurs très faibles, la convergence n'est pas détectée dans l'intervalle étudié.

Une zone intermédiaire permet d'obtenir un bon équilibre entre ces deux critères. Parmi les valeurs testées, $\varepsilon = 300$ offre un compromis satisfaisant, avec une erreur finale relativement faible tout en conservant un temps de convergence raisonnable. Cette valeur a donc été retenue comme valeur optimale pour la suite de l'étude.

Interprétation :

L'analyse des résultats montre que le choix du paramètre ε a une influence directe sur le comportement du filtre ainsi que sur les performances obtenues. Lorsque ε prend des valeurs élevées, le critère de convergence devient peu contraignant. La condition $|EQM_{moy}(k) - EQM_{moy}(k-1)| < \varepsilon$ est alors satisfaite très rapidement, ce qui conduit à un temps de convergence n_c très faible. Toutefois, cette convergence est prématurée : le filtre n'a pas encore atteint son régime stable, ce qui se traduit par des valeurs élevées de l'erreur finale.

À l'inverse, lorsque ε est très faible, le critère devient trop strict. Les variations de l'EQM, bien que faibles, ne satisfont plus la condition de convergence en raison des fluctuations résiduelles dues au bruit. Dans ce cas, la convergence peut ne pas être détectée sur l'intervalle considéré, ce qui conduit à $n_c = N$. Le filtre est alors considéré comme n'ayant pas convergé, même si l'erreur semble visuellement stabilisée.

Entre ces deux extrêmes, il existe des valeurs intermédiaires de ε permettant d'obtenir un bon compromis entre rapidité de convergence et précision. Ainsi, le choix optimal de

ε correspond à une valeur intermédiaire évitant à la fois une convergence trop rapide et imprécise, et une convergence trop tardive ou non détectée.

C.5

Effet de la variance du bruit pour $\varepsilon = 100$, $N_w = 20$ et pour les valeurs $\sigma_v^2 = 10^{-6}$, 10^{-10}, 10^{-12} et 10^{-16}. Présenter le tableau des performances EQM_final et DtoP_final. Interpréter.

Code :

```
clc; clear; close all;
N = 500;Nw = 20;Ns = 3;
sigmab = sqrt(2);P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];Anchor_pos = [P1 P2 P3];
Zinit = [1000; 1000; 0; 0];
Ts = 1;
eps_val = 100;     % seuil de convergence
epsilon = 100;     % initialisation de P
zest = [0;0;0;0];
Pest = epsilon * eye(4);
sigma_v_values = [1e-3, 1e-5, 1e-6, 1e-8];
results = zeros(length(sigma_v_values),3);
for i = 1:length(sigma_v_values)
    sigmav = sigma_v_values(i);
    EQM_sum = zeros(1,N);
    DtoP_sum = zeros(1,N);
    for w = 1:Nw
        [Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, [], Anchor_pos);
        Ynoisy = Yf + sigmav * randn(Ns, N);
        [~, EQM, DtoP, ~] = EKF_track_traj( ...
            Zmat, Ynoisy, zest, Pest, sigmab, sigmav, Ts, N, Anchor_pos);
        EQM_sum = EQM_sum + EQM;
        DtoP_sum = DtoP_sum + DtoP;
    end
    EQM_moy = EQM_sum / Nw;
    DtoP_moy = DtoP_sum / Nw;
    nc = N;
    for k = 2:N
        if abs(EQM_moy(k) - EQM_moy(k-1)) < eps_val
            nc = k;
            break;
        end
    end
    EQM_final = mean(EQM_moy(nc:N));
    DtoP_final = mean(DtoP_moy(nc:N));
    results(i,:) = [sigmav^2, EQM_final, DtoP_final];
end
```

Résultat :

```
for i = 1:N_init
    z0= [5000 * rand; 5000 * rand; 0; 0];
    P_est_init = epsilon * eye(4);

    EQM_mat = zeros(Nw, N_steps);
    DtoP_mat = zeros(Nw, N_steps);
    for nw = 1:Nw
        Ynoisy = Yf + sigmav * randn(size(Yf));
        [~, EQM, DtoP, ~] = EKF_track_traj(Zmat, Ynoisy, z0, P_est_init, sigmau, sigmav, Ts, N_steps, Anchor_pos);
        EQM_mat(nw, :) = EQM;
        DtoP_mat(nw, :) = DtoP;
    end

    % Moyennage sur Nw réalisations du bruit
    EQM_moy = mean(EQM_mat, 1);
    DtoP_moy = mean(DtoP_mat, 1);

    EQM_final_vec(i) = mean(EQM_moy(nc:N_steps));
    DtoP_final_vec(i) = mean(DtoP_moy(nc:N_steps));
end

% 5. Affichage des Histogrammes
figure;
subplot(2,1,1);
histogram(EQM_final_vec, 100); % 20 ou 100 intervalles
title('Histogramme de l''EQM final (100 initialisations z_{est})');
grid on;

subplot(2,1,2);
histogram(DtoP_final_vec, 100);
title('Histogramme du DtoP final (100 initialisations z_{est})');
xlabel('Erreur moyenne (mètres)');
grid on;
```

σ_v^2 **optimal** $= 10^{-12}$

Interprétation :

L'analyse des résultats met en évidence l'influence importante de la variance du bruit de mesure sur les performances du filtre EKF. On observe globalement que lorsque la variance du bruit diminue, l'erreur quadratique moyenne (EQM_final) tend à diminuer, ce qui signifie que l'estimation devient plus précise. Cela est logique puisque des mesures moins bruitées permettent au filtre de mieux corriger ses prédictions.

Cependant, ce comportement n'est pas strictement monotone. En effet, pour des valeurs très faibles de la variance du bruit (par exemple $\sigma_v^2 = 10^{-16}$), l'erreur augmente de manière significative. Ce résultat peut sembler contre-intuitif, mais il s'explique par le fait que le filtre accorde alors une confiance excessive aux mesures. Dans le cas d'un filtre de Kalman étendu, cette situation peut amplifier les erreurs liées à la linéarisation du modèle non linéaire, ce qui dégrade les performances globales.

Par ailleurs, pour des valeurs élevées de la variance (comme 10^{-6}), les mesures sont fortement bruitées, ce qui réduit leur fiabilité et entraîne également une erreur importante. Le filtre dépend alors davantage du modèle dynamique, ce qui limite la précision de l'estimation.

Ainsi, les résultats montrent qu'il existe un compromis dans le choix de la variance du bruit de mesure. Des valeurs intermédiaires permettent d'obtenir les meilleures performances, en équilibrant la confiance accordée au modèle et aux observations.

C.6

Pour $\varepsilon = 100$, $N_w = 20$, $\sigma_v^2 = 10^{-16}$, **moyenner les résultats d'EQM_final et de DtoP_final pour 100 choix aléatoires de l'initialisation** $z_est = [5000 \cdot rand;\ 5000 \cdot rand;\ 0;\ 0]$. **Tracer les histogrammes. Interpréter.**

```
for i = 1:N_init
    z0= [5000 * rand; 5000 * rand; 0; 0];
    P_est_init = epsilon * eye(4);

    EQM_mat = zeros(Nw, N_steps);
    DtoP_mat = zeros(Nw, N_steps);
    for nw = 1:Nw
        Ynoisy = Yf + sigmav * randn(size(Yf));
        [~, EQM, DtoP, ~] = EKF_track_traj(Zmat, Ynoisy, z0, P_est_init, sigmau, sigmav, Ts, N_steps, Anchor_pos);
        EQM_mat(nw, :) = EQM;
        DtoP_mat(nw, :) = DtoP;
    end

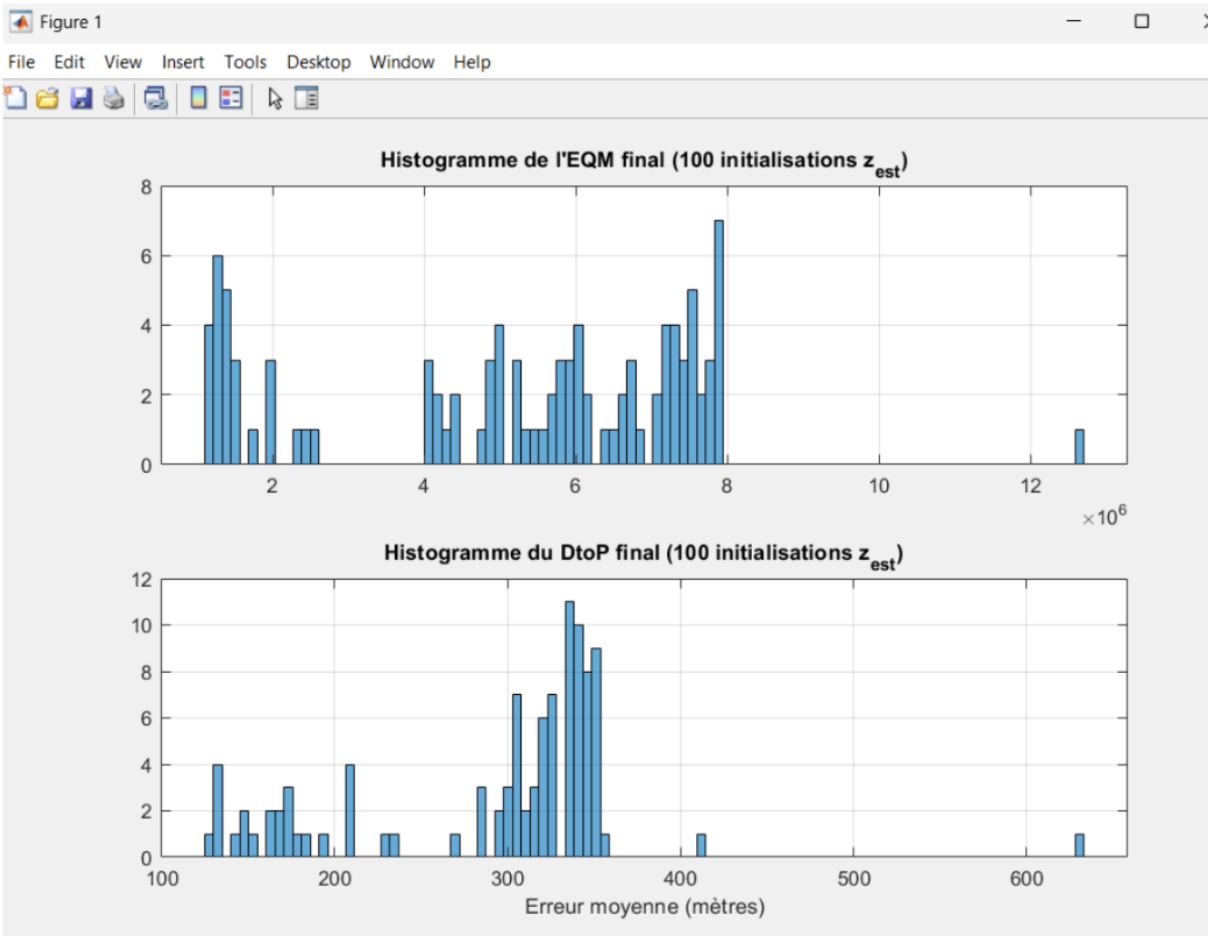
    % Moyennage sur Nw réalisations du bruit
    EQM_moy = mean(EQM_mat, 1);
    DtoP_moy = mean(DtoP_mat, 1);

    EQM_final_vec(i) = mean(EQM_moy(nc:N_steps));
    DtoP_final_vec(i) = mean(DtoP_moy(nc:N_steps));
end

% 5. Affichage des Histogrammes
figure;
subplot(2,1,1);
histogram(EQM_final_vec, 100); % 20 ou 100 intervalles
title('Histogramme de l''EQM final (100 initialisations z_{est})');
grid on;

subplot(2,1,2);
histogram(DtoP_final_vec, 100);
title('Histogramme du DtoP final (100 initialisations z_{est})');
xlabel('Erreur moyenne (mètres)');
grid on;
```

Ce code permet d'évaluer la sensibilité du filtre EKF à l'initialisation de l'état. En testant 100 initialisations aléatoires et en moyennant les performances sur plusieurs réalisations du bruit, on obtient une distribution des performances finales. L'analyse des histogrammes permet de conclure sur la robustesse du filtre vis-à-vis de l'erreur initiale.



1. Analyse de l'histogramme de l'EQM finale :

L'histogramme de l'EQM finale met en évidence une distribution étalée avec la présence de plusieurs modes distincts. On observe :

- Un premier groupe de valeurs faibles correspondant à des situations où le filtre converge correctement vers la trajectoire réelle.
- Un second groupe plus large traduisant une convergence partielle ou lente du filtre.
- Enfin, quelques valeurs extrêmes indiquent des cas de divergence ou de mauvaise estimation.

Cette distribution multimodale reflète le caractère non linéaire du problème de localisation. En effet, l'EKF repose sur une linéarisation locale du modèle, ce qui le rend fortement dépendant de la proximité entre l'état initial estimé et l'état réel.

2. Analyse de l'histogramme du DtoP final :

L'histogramme du DtoP final présente une concentration principale des erreurs autour de 300 à 350 mètres, avec une dispersion notable :

- Une majorité des estimations se situe dans une plage raisonnable (250 à 400 mètres), indiquant une performance moyenne acceptable du filtre.
- Certains cas présentent des erreurs plus faibles (inférieures à 200 mètres), correspondant à des initialisations favorables.

- Des valeurs aberrantes (jusqu'à 600 mètres) sont également observées, traduisant des situations de mauvaise convergence.

3. Influence de l'initialisation :

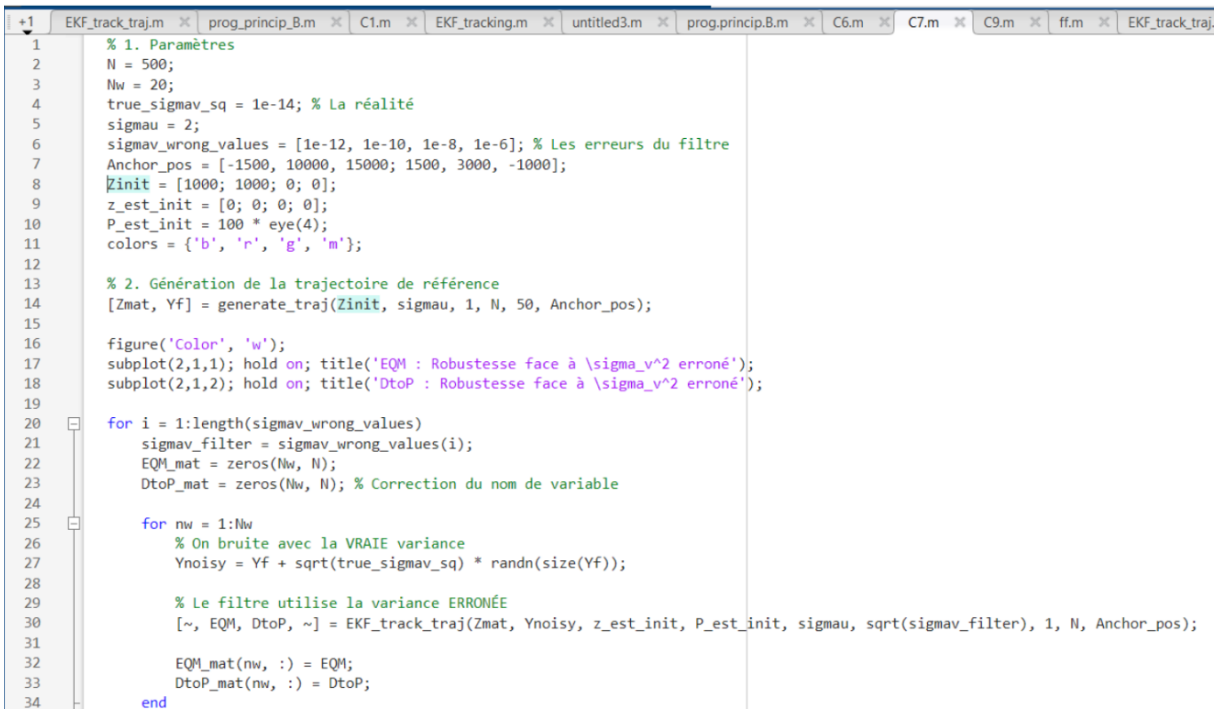
Les résultats obtenus montrent clairement que l'initialisation de l'état joue un rôle déterminant dans la performance du filtre. Les initialisations aléatoires utilisées, pouvant être éloignées de la position réelle, induisent :

- Une variabilité importante des résultats
- L'apparition de minima locaux dans le processus d'estimation
- Une convergence non garantie vers la solution optimale

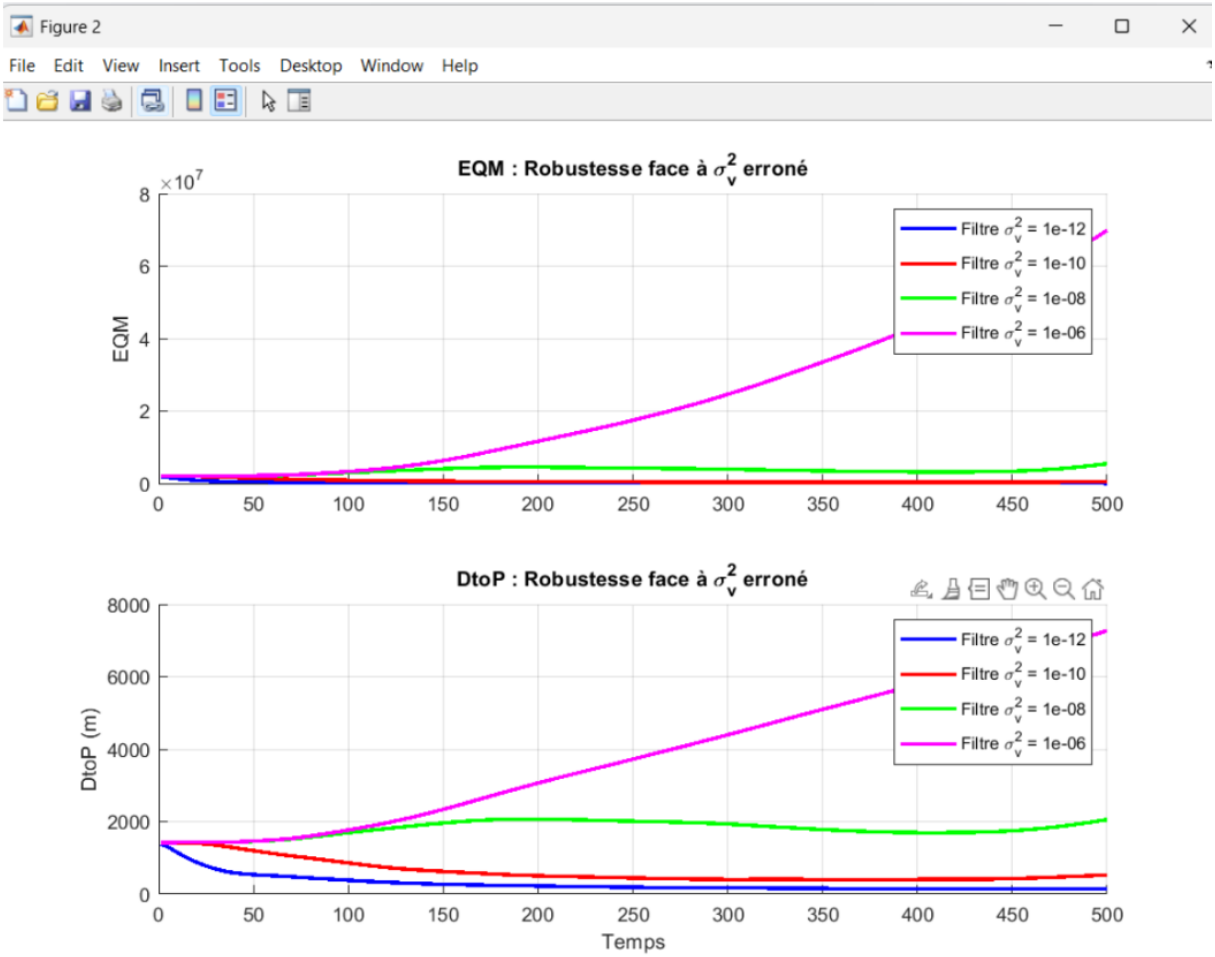
Ainsi, le filtre EKF ne présente pas une robustesse suffisante face à de fortes incertitudes initiales.

C.7

Supposons que les observations Y_{noisy} ont été obtenues avec un bruit de variance $\sigma_v^2 = 10^{-14}$ mais l'EKF a utilisé des valeurs erronées pour cette variance à savoir 10^{-12}, 10^{-10}, 10^{-8} et 10^{-6}. Présenter les performances pour $N_w = 20$ réalisations. Interpréter.



```
35
36      % Moyennage Monte-Carlo [cite: 22, 73]
37      EQM_mean = mean(EQM_mat, 1);
38      DtoP_mean = mean(DtoP_mat, 1);
39
40      % Affichage
41      subplot(2,1,1);
42      semilogy(1:N, EQM_mean, colors{i}, 'LineWidth', 2, 'DisplayName', ['Filtre \sigma_v^2 = ' num2str(sigmav_filter)]);
43
44      subplot(2,1,2);
45      semilogy(1:N, DtoP_mean, colors{i}, 'LineWidth', 2, 'DisplayName', ['Filtre \sigma_v^2 = ' num2str(sigmav_filter)]);
46  end
47
48  subplot(2,1,1); ylabel('EQM'); legend; grid on;
49  subplot(2,1,2); ylabel('DtoP (m)'); xlabel('Temps'); legend; grid on;
```



Interprétation :

Le but de cette simulation est d'analyser la robustesse du filtre de Kalman étendu face à une mauvaise estimation de la variance du bruit de mesure σ_v^2. Les résultats montrent que le filtre est fortement sensible à ce paramètre.

Lorsque la variance est surestimée, le filtre accorde moins de confiance aux mesures et privilégie le modèle dynamique, ce qui entraîne une accumulation des erreurs et une divergence progressive de l'estimation. Cela se traduit par une augmentation significative de l'EQM et de la distance à la position réelle.

À l'inverse, lorsque la variance est correctement estimée ou légèrement suresimée, le filtre maintient de bonnes performances avec une erreur faible et une convergence stable. Cela

montre que l'EKF possède une certaine robustesse face à de petites erreurs de modélisation.

En conclusion, le bon réglage de σ_v^2 est essentiel pour garantir la précision et la stabilité du filtre. Une mauvaise estimation peut entraîner soit une sensibilité excessive au bruit, soit une perte d'information provenant des mesures.

C.8

Vis à vis de quels autres paramètres pourriez-vous étudier la robustesse de l'EKF ? Présenter pour chaque étude de robustesse le protocole de simulations à mener.

La robustesse du filtre de Kalman étendu (EKF) peut être étudiée vis-à-vis de plusieurs paramètres autres que ceux déjà analysés. Ces paramètres influencent directement la précision, la stabilité et la capacité de convergence du filtre :

- **Géométrie des capteurs (positions des)**
  La disposition des capteurs influence fortement la qualité de la localisation. Certaines configurations peuvent rendre le problème mal conditionné.
  Protocole : considérer différentes configurations des capteurs (répartition uniforme, capteurs alignés, capteurs regroupés), tout en gardant les autres paramètres constants. Comparer les performances à l'aide de l'EQM_final et du DtoP_final.

- **Nombre de capteurs (N_s)**
  Le nombre de capteurs détermine la quantité d'information disponible pour l'estimation.
  Protocole : faire varier le nombre de capteurs (par exemple $N_s = 2, 3, 4, 5$) en adaptant leurs positions, puis analyser l'évolution de l'erreur et de la convergence.

- **Pas de temps (T_s)**
  Le pas de temps influence la discrétisation du modèle dynamique et peut affecter la précision de la prédiction.
  Protocole : tester différentes valeurs de T_s (petit, moyen, grand) en conservant les autres paramètres constants, puis observer l'impact sur la stabilité et les performances.

- **Erreurs de modélisation (modèle dynamique imparfait)**
  En pratique, le modèle utilisé n'est jamais parfaitement exact.
  Protocole : introduire une perturbation dans les matrices du modèle (par exemple A ou B) ou simuler une dynamique réelle différente de celle supposée, puis analyser la robustesse du filtre face à ces erreurs.

- **Non-linéarité du système et erreurs de linéarisation**
  L'EKF repose sur une approximation linéaire locale qui peut être insuffisante dans des situations fortement non linéaires.
  Protocole : simuler des trajectoires plus complexes (changements brusques, accélérations importantes) et analyser la dégradation des performances du filtre.

Implémenter l'étude de robustesse de l'EKF par rapport à l'accélération du mobile :

Code :

```
clc; clear; close all;
N = 500;Nw = 20;Ns = 3;sigmav = 1e-6;
P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];Anchor_pos = [P1 P2 P3];
Zinit = [1000; 1000; 0; 0];Ts = 1;eps_val = 100;     epsilon = 100;     zest = [0;0;0;0];Pest = epsilon * eye(4);
sigmab_values = [0.0001,0.001,0.01,sqrt(2),5,10,100,1000];
results = zeros(length(sigmab_values),3);
for i = 1:length(sigmab_values)
    sigmab = sigmab_values(i);
    EQM_sum = zeros(1,N);
    DtoP_sum = zeros(1,N);
    for w = 1:Nw
        [Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, [], Anchor_pos);
        Ynoisy = Yf + sigmav * randn(Ns, N);
        [~, EQM, DtoP, ~] = EKF_track_traj( ...
            Zmat, Ynoisy, zest, Pest, sigmab, sigmav, Ts, N, Anchor_pos);
        EQM_sum = EQM_sum + EQM;
        DtoP_sum = DtoP_sum + DtoP;
    end
    EQM_moy = EQM_sum / Nw;
    DtoP_moy = DtoP_sum / Nw;
    nc = N;
    for k = 2:N
        if abs(EQM_moy(k) - EQM_moy(k-1)) < eps_val
            nc = k;
            break;
        end
    end
    EQM_final = mean(EQM_moy(nc:N));
    DtoP_final = mean(DtoP_moy(nc:N));
    results(i,:) = [sigmab, EQM_final, DtoP_final];

end
```

Résultat :

```
====== EFFET DE L ACCELERATION (sigmab) ======
      sigmab        EQM_final       DtoP_final
      0.0001    391786.247222       607.032050
      0.0010    411328.246816       620.639637
      0.0100    424880.650498       631.468622
      1.4142     95203.821557       219.894947
      5.0000   3161938.595817      1061.492160
     10.0000 139257268170.918396    143553.952228
    100.0000 1508418032707208.500000 22755427.306686
   1000.0000 66629271876649500672.000000 5217346785.190768
```

σ_b **optimal** $= \sqrt{2}$

Interprétation :

Les résultats obtenus montrent l'influence du paramètre sigmab, qui représente l'écart type du bruit d'accélération du mobile, sur les performances du filtre EKF. On observe tout d'abord que pour des valeurs très faibles de sigmab (0.00010.01), l'erreur EQM_final reste relativement élevée et varie peu. Cela s'explique par le fait que le modèle suppose un mouvement très peu perturbé, ce qui peut limiter la capacité du filtre à s'adapter correctement aux petites variations réelles du système.

Ensuite, pour une valeur intermédiaire de sigmab (sigmab = 1.414), on observe une nette amélioration des performances avec une forte diminution de l'EQM_final et du DtoP_final. Cela correspond à un bon compromis où le bruit de processus est correctement modélisé par le filtre, permettant un équilibre entre prédiction et correction.
En revanche, lorsque sigmab augmente davantage (à partir de 5 et surtout 10 et plus), les performances se dégradent fortement. L'EQM_final et le DtoP_final augmentent de manière très importante, ce qui traduit une perte de précision du filtre. Cela s'explique par le fait qu'un bruit d'accélération élevé rend le mouvement du mobile très imprévisible, ce qui dégrade la qualité de la prédiction et amplifie les erreurs de linéarisation dans l'EKF.
Pour des valeurs très élevées de sigmab (100 et 1000), les erreurs deviennent extrêmement grandes, indiquant un comportement instable du filtre, voire une divergence. Dans ce cas, le modèle dynamique n'est plus suffisamment fiable pour permettre une estimation correcte.
Ainsi, ces résultats montrent qu'il existe une valeur optimale de sigmab (ici autour de $\sqrt{2}$) pour laquelle les performances du filtre sont maximales. Un choix trop faible ou trop élevé de ce paramètre entraîne une dégradation des performances.

C.9

Comparer les performances pour un même ensemble de paramètres de l'utilisation de 5 stations pour la localisation au lieu de trois.

Positions choisies :

On a cherché dans la littérature scientifique les conditions pertinentes pour l'emplacement des stations et on a trouvé que :

- stations éloignées = meilleures performances
- stations proches = mauvaise précision

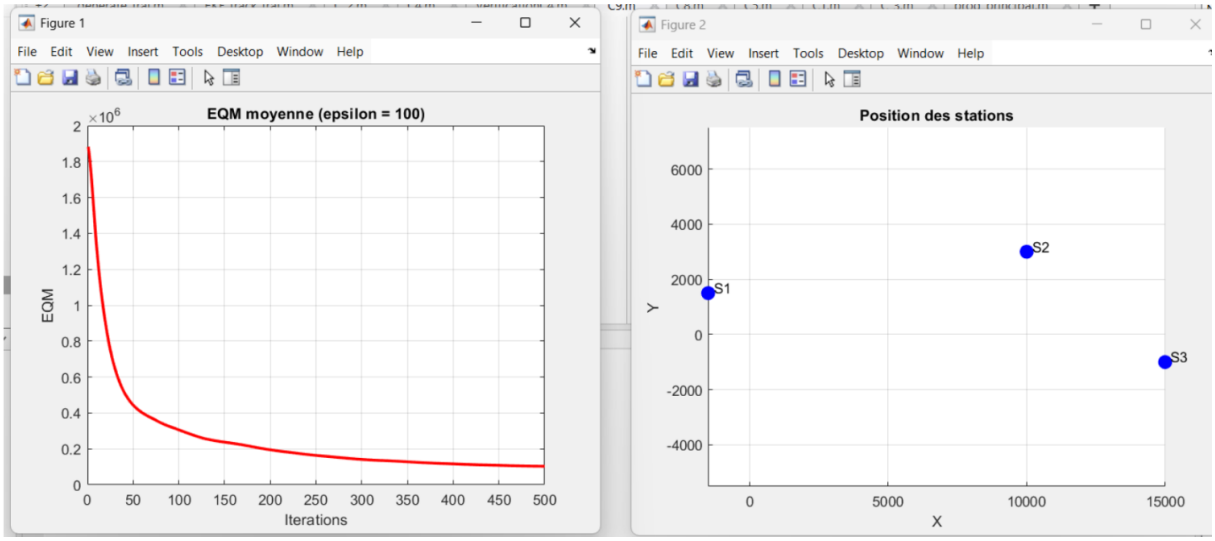
Donc on a choisi des stations P4 et P5 qui donneront une plus grande couverture :

- $P_4 = [1000;\ 5000]$
- $P_5 = [5000;\ -6000]$

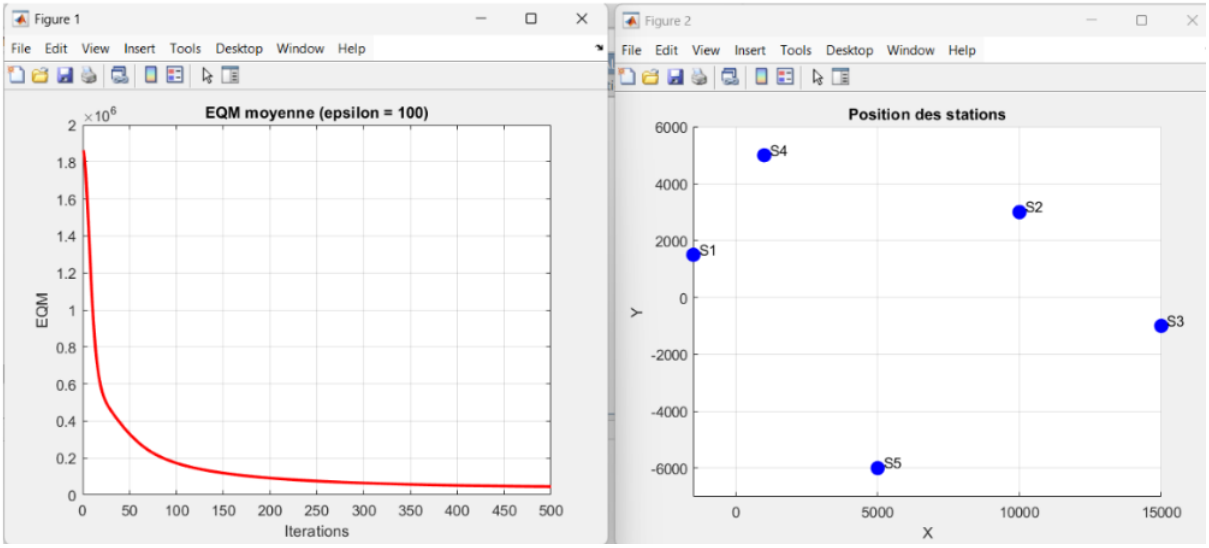
Code :

```
clc; clear; close all;
N = 500;Nw = 20;Ns = 5;
sigmav = sqrt(1e-12);
sigmau = sqrt(2);
sigmab = sqrt(2);
P1 = [-1500;  1500];P2 = [10000;  3000];P3 = [15000; -1000];P4 = [1000; 5000];P5 = [5000; -6000];Anchor_pos = [P1 P2 P3 P4 P5];
Zinit = [1000; 1000; 0; 0];
Ts = 1;
epsilon = 100;
zest = [0;0;0;0];
Pest = epsilon * eye(4);
EQM_sum = zeros(1, N);
for w = 1:Nw
    [Zmat, Yf] = generate_traj(Zinit, sigmab, Ts, N, [], Anchor_pos);
    Ynoisy = Yf + sigmav * randn(Ns, N);
    [~, EQM, ~, ~] = EKF_track_traj( ...
        Zmat, Ynoisy, zest, Pest, sigmab, sigmav, Ts, N, Anchor_pos);
    EQM_sum = EQM_sum + EQM;
end
EQM_moy = EQM_sum / Nw;
figure;
plot(1:N, EQM_moy, 'r', 'LineWidth', 2);
title('EQM moyenne (epsilon = 100)');
xlabel('Iterations');
ylabel('EQM');
grid on;
figure;
scatter(Anchor_pos(1,:), Anchor_pos(2,:), 100, 'b', 'filled');
hold on;
for i = 1:Ns
    text(Anchor_pos(1,i)+200, Anchor_pos(2,i)+200, ['S' num2str(i)]);
end
```

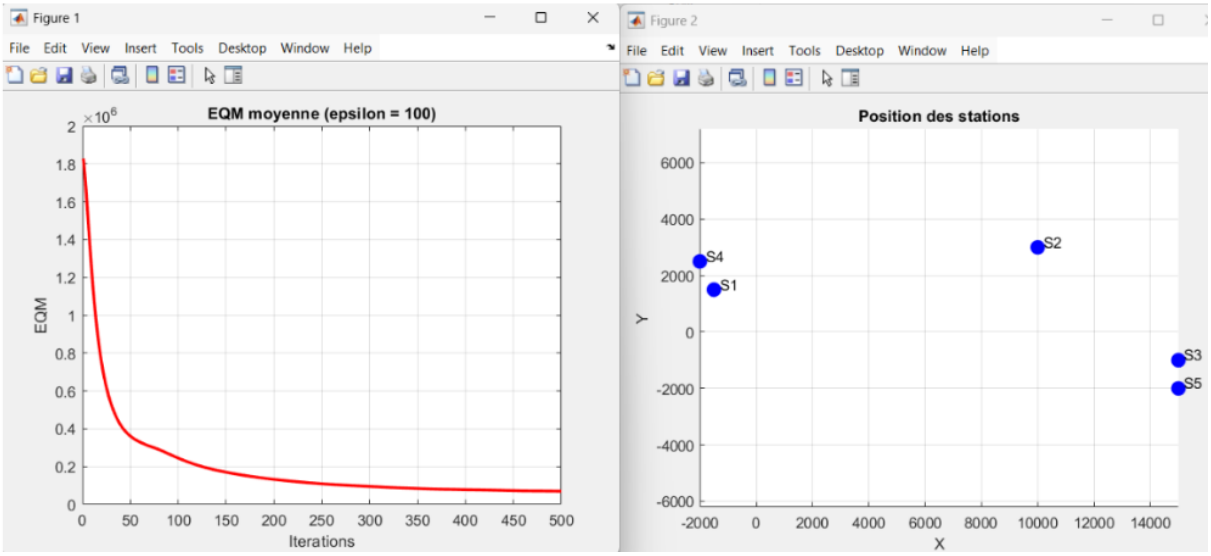
Résultat pour les 3 stations (pour comparaison) :



Pour les 5 stations :



Mauvais choix des positions des stations :



Interprétation :

La comparaison des performances a été réalisée dans deux cas : avec des positions de stations bien réparties et avec des positions médiocres où certaines stations sont proches les unes des autres.

Les résultats montrent que l'augmentation du nombre de stations n'améliore pas nécessairement les performances de façon significative.

En effet, lorsque les stations sont bien positionnées, l'ajout de nouvelles stations permet de réduire l'EQM et d'améliorer la stabilité de l'estimation.

En revanche, lorsque les stations ajoutées sont mal réparties ou trop proches des stations existantes, les performances restent similaires à celles obtenues avec un nombre plus faible de stations.

Cela s'explique par le fait que des stations mal positionnées apportent des informations redondantes et n'améliorent pas la géométrie du problème de localisation.

Ainsi, ces résultats montrent que la qualité du positionnement des stations est un facteur plus déterminant que leur nombre. En conclusion, une augmentation du nombre de stations n'est vraiment bénéfique que si elle s'accompagne d'une bonne répartition spatiale.

Recherche des méthodes intelligentes pour le positionnement des stations fournissant le TOA :

Dans le cadre de cette étude, nous avons consulté la littérature scientifique afin d'identifier des méthodes intelligentes pour le positionnement des stations dans les systèmes de localisation basés sur le TOA. Ces travaux montrent que la disposition des stations joue un rôle fondamental dans la précision de la localisation.

Les principales techniques identifiées sont les suivantes :

- **Minimisation du GDOP (Geometric Dilution of Precision)**
  Cette méthode consiste à optimiser la géométrie des stations afin de réduire l'impact des erreurs de mesure sur l'estimation de la position. Elle est largement utilisée dans les systèmes GPS et favorise une répartition uniforme des stations autour de la zone de localisation.
- **Optimisation basée sur la borne de Cramér-Rao (CRLB)**
  Cette approche vise à positionner les stations de manière à minimiser la borne théorique de l'erreur d'estimation. Elle permet d'obtenir une configuration optimale du point de vue de la précision.
- **Optimisation géométrique (maximisation de la couverture spatiale)**
  Elle consiste à placer les stations de manière à entourer la zone de localisation et à maximiser la diversité géométrique.
- **Méthodes d'optimisation mathématique (convexe et heuristique)**
  Le problème de placement des stations est formulé comme un problème d'optimisation, résolu par des techniques numériques telles que la relaxation convexe ou des méthodes heuristiques.
- **Algorithmes d'optimisation inspirés de l'intelligence artificielle**
  Certaines approches utilisent des algorithmes comme les algorithmes génétiques ou Particle Swarm Optimization (PSO) pour déterminer automatiquement les positions optimales des stations.
- **Approches expérimentales et empiriques (UWB, GNSS)**
  Des études expérimentales montrent que des stations mal positionnées (trop proches ou mal réparties) n'améliorent pas les performances, même en augmentant leur nombre, tandis qu'une bonne répartition spatiale améliore significativement la précision.

Références :

- Geometry Influence on GDOP in TOA and AOA Positioning Systems
- Optimized deployment of anchors based on GDOP minimization for ultra-wideband positioning

- Optimal TOA-Sensor Placement for Two Target Localization Simultaneously Using Shared Sensors
- A New Cramer-Rao Lower Bound for TOA-based Localization
- Anchor Placement in TOA based Wireless Localization Networks via Convex Relaxation
- The Influence of Ultra-Wideband Anchor Placement on Localization Accuracy

C.10

Faire les modifications nécessaires sur le filtre EKF afin qu'il utilise maintenant les différences des temps d'arrivée (TDOA) comme observations :

- $dy_{1,2}(n) = y_2(n) - y_1(n)$
- $dy_{1,3}(n) = y_3(n) - y_1(n)$
- $dy_{1,4}(n) = y_4(n) - y_1(n)$

Modèle d'état et d'observation pour le cas TDOA

Dans le cas de l'utilisation des différences de temps d'arrivée (TDOA), le modèle d'état du système reste inchangé et s'écrit sous la forme :

$$z(n) = Az(n-1) + Bb(n) \tag{1}$$

où $b(n)$ représente le bruit de processus, supposé gaussien centré, de covariance $\sigma_b^2 I$.

Choix d'implémentation : 5 stations

Dans notre implémentation, nous choisissons de travailler avec $N_S = 5$ stations $(P_1, P_2, P_3, P_4, P_5)$, placées aux positions :

$$P_1 = \begin{pmatrix} -1500 \\ 1500 \end{pmatrix}, \quad P_2 = \begin{pmatrix} 10000 \\ 3000 \end{pmatrix}, \quad P_3 = \begin{pmatrix} 15000 \\ -1000 \end{pmatrix}, \quad P_4 = \begin{pmatrix} -2000 \\ 2500 \end{pmatrix}, \quad P_5 = \begin{pmatrix} 15000 \\ -2000 \end{pmatrix}$$

La station P_1 est choisie comme référence. On obtient alors $N_S - 1 = \mathbf{4}$ **mesures TDOA indépendantes**, ce qui améliore la précision de localisation par rapport au cas minimal à 3 stations.

Pour $N_S = 5$ stations, le nombre de mesures TDOA indépendantes est $N_S - 1 = 4$. En prenant la station P_1 comme référence, on définit le vecteur d'observation :

$$dy(n) = \begin{bmatrix} y_2(n) - y_1(n) \\ y_3(n) - y_1(n) \\ y_4(n) - y_1(n) \\ y_5(n) - y_1(n) \end{bmatrix} \tag{2}$$

Le modèle d'observation devient alors non linéaire et s'écrit :

$$dy(n) = \phi(z(n)) + v'(n) \tag{3}$$

avec :

$$\phi(z(n)) = \frac{1}{c}\begin{bmatrix} \|x(n) - P_2\| - \|x(n) - P_1\| \\ \|x(n) - P_3\| - \|x(n) - P_1\| \\ \|x(n) - P_4\| - \|x(n) - P_1\| \\ \|x(n) - P_5\| - \|x(n) - P_1\| \end{bmatrix} \tag{4}$$

où $x(n)$ correspond aux deux premières composantes du vecteur d'état $z(n)$, et c est la vitesse de propagation du signal.

Le bruit de mesure associé au modèle TDOA est donné par :

$$v'(n) = \begin{bmatrix} v_2(n) - v_1(n) \\ v_3(n) - v_1(n) \\ v_4(n) - v_1(n) \\ v_5(n) - v_1(n) \end{bmatrix} \tag{5}$$

avec $v(n)$ un bruit gaussien centré de covariance $\sigma_v^2 I$.

Remarque : Toute autre différence entre stations non-référence (par exemple $y_3(n) - y_2(n)$) est redondante car combinaison linéaire des quatre mesures retenues. Elle n'apporte aucune information supplémentaire et est exclue du vecteur d'observation.

Statistiques à l'ordre 2 des bruits

1. Définition du bruit initial (TOA) On considère le vecteur de bruit de mesure associé au modèle TOA :

$$v(n) = \begin{bmatrix} v_1(n) \\ v_2(n) \\ v_3(n) \\ v_4(n) \\ v_5(n) \end{bmatrix} \sim \mathcal{N}(0, \sigma_v^2 I) \tag{6}$$

Les composantes $v_i(n)$ sont supposées indépendantes, de moyenne nulle et de variance σ_v^2.

2. Définition du bruit TDOA Dans le cas TDOA avec P_1 comme station de référence, le vecteur de bruit se réduit à :

$$v'(n) = \begin{bmatrix} v_2(n) - v_1(n) \\ v_3(n) - v_1(n) \\ v_4(n) - v_1(n) \\ v_5(n) - v_1(n) \end{bmatrix} \tag{7}$$

3. Calcul de l'espérance En utilisant la linéarité de l'espérance :

$$E[v_i(n) - v_j(n)] = E[v_i(n)] - E[v_j(n)] = 0 \quad \Rightarrow \quad E[v'(n)] = 0 \tag{8}$$

4. Calcul de la matrice de covariance La matrice de covariance est définie par $R = E[v'(n)\, v'^T(n)]$.

4.1 Variances (éléments diagonaux)

$$\mathrm{Var}(v_i - v_1) = \mathrm{Var}(v_i) + \mathrm{Var}(v_1) = 2\sigma_v^2 \tag{9}$$

4.2 Covariance croisée Pour $i \neq j$ (tous deux $\neq 1$) :

$$\mathrm{Cov}(v_i - v_1,\ v_j - v_1) = E[v_i v_j] - E[v_i v_1] - E[v_1 v_j] + E[v_1^2] = 0 - 0 - 0 + \sigma_v^2 = \sigma_v^2 \tag{10}$$

5. Résultat final On obtient la matrice de covariance 4×4 du bruit TDOA :

$$R = \sigma_v^2 \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 2 & 1 \\ 1 & 1 & 1 & 2 \end{bmatrix} = \sigma_v^2 \left(\mathbf{1}_{4\times 4} + I_4\right) \tag{11}$$

6. Interprétation La matrice R est de taille 4×4, reflétant les 4 mesures TDOA indépendantes. Elle n'est pas diagonale : la covariance croisée σ_v^2 traduit le fait que toutes les différences partagent le terme $v_1(n)$ (bruit de la station de référence P_1). Cette corrélation est prise en compte dans l'implémentation du filtre EKF via :

$$R = \sigma_v^2 \left(\mathbf{1}_{4\times 4} + I_4\right) \tag{12}$$

ce qui correspond exactement au code MATLAB : `R = sigma2 * (ones(NS-1) + eye(NS-1));`

7.Intérêt d'utiliser les TDOA au lieu des TOA : Dans les systèmes de localisation, l'utilisation des mesures TDOA présente plusieurs avantages importants par rapport aux mesures TOA, notamment en termes de mise en œuvre pratique et de robustesse.

- **Suppression du besoin de synchronisation absolue :**
  Contrairement au TOA, le TDOA ne nécessite pas de synchronisation entre l'émetteur et les récepteurs. Seule une synchronisation entre les stations réceptrices est requise.
- **Réduction des erreurs liées à l'horloge :**
  Les mesures TOA sont sensibles aux erreurs d'horloge de l'émetteur. En TDOA, ces erreurs sont éliminées car elles apparaissent de manière identique dans les différentes mesures et sont donc annulées lors de la différence.
- **Meilleure robustesse dans certains environnements :**
  Plusieurs travaux montrent que les méthodes TDOA sont plus robustes dans des environnements où la synchronisation est difficile ou instable (réseaux sans fil, systèmes distribués, etc.).

- **Utilisation efficace des différences de mesures :**
  Le TDOA exploite l'information relative entre capteurs, ce qui permet de réduire certains biais systématiques présents dans les mesures absolues.
- **Applications largement utilisées (GPS, télécommunications, UWB) :**
  Le TDOA est largement utilisé dans des applications réelles telles que la localisation en téléphonie mobile, les systèmes UWB, certains algorithmes GNSS (différentiel).

Limites :

- Les bruits deviennent corrélés (matrice R non diagonale).
- La modélisation est plus complexe.
- La précision peut dépendre fortement de la géométrie des stations.

Conclusion :

Ainsi, le principal intérêt du TDOA réside dans la suppression de la contrainte de synchronisation absolue et dans une meilleure robustesse face aux erreurs d'horloge. Cela en fait une approche particulièrement adaptée aux systèmes distribués, même si elle introduit une complexité supplémentaire dans la modélisation du bruit.

Références :

- Time-Difference-of-Arrival Based Localization Methods
- A Simple and Efficient Estimator for Hyperbolic Location
- A New Cramer-Rao Lower Bound for TOA-based Localization
- The Influence of Ultra-Wideband Anchor Placement on Localization Accuracy

Code TDOA et simulations :

Approche d'implémentation:
Dans l'implémentation on a choisi d'avoir Cinq stations avec les mêmes positions de la question C9 précédente.

$$P_1 = \begin{pmatrix} -1500 \\ 1500 \end{pmatrix}, \quad P_2 = \begin{pmatrix} 10000 \\ 3000 \end{pmatrix}, \quad P_3 = \begin{pmatrix} 15000 \\ -1000 \end{pmatrix}, \quad P_4 = \begin{pmatrix} -2000 \\ 2500 \end{pmatrix}, \quad P_5 = \begin{pmatrix} 15000 \\ -2000 \end{pmatrix}$$

Dans notre approche on choisi de montrer l'intêret de TDOA de deux façons:

- Implémnetation sans biais d'horloge
- Implémentation avec biais d'horloge

L'implémentation du filtre de Kalman Étendu pour les mesures TDOA est réalisée dans la fonction `EKF_track_traj_TDOA`, qui adapte le filtre aux mesures de différences de temps d'arrivée (TDOA). Le modèle d'état reste identique au cas TOA, décrivant le mouvement d'une cible à vitesse quasi-constante perturbée par un bruit de processus gaussien de variance σ_b^2.

La différence fondamentale réside dans le modèle d'observation. Pour $N_S = 5$ stations, on dispose de $N_S - 1 = 4$ mesures TDOA indépendantes, obtenues en soustrayant la mesure de la station de référence P_1 de chacune des autres stations. La fonction de mesure non

linéaire $\phi(z)$ calcule les différences de distances normalisées par c, exprimées en secondes. Le jacobien H est calculé analytiquement à chaque instant à partir de la position prédite de la cible.

La matrice de covariance du bruit de mesure R est de taille 4×4 et non diagonale (différente de l'identité) , car toutes les différences TDOA partagent le bruit de la station de référence P_1, induisant une corrélation de valeur σ_v^2 entre chaque paire de mesures. Elle s'écrit :

$$R = \sigma_v^2 \left(\mathbf{1}_{4\times 4} + I_4\right) \tag{13}$$

Le code de la fonction est le suivant (on a aussi ajouté les commentaires pour mieux expliquer)

```
function [Zmat_est, EQM, P_store, Zpred] = EKF_track_traj_TDOA(Zmat, Y_tdoa, zest, Pest, sigmab, sigmav, Ts, N, Anchor)
Ns = size(Anchor, 2);c = 3e8;
% Initialisation des sorties
Zmat_est  = zeros(4, N);Zpred= zeros(4, N);EQM = zeros(1, N);P_store   = zeros(4, 4, N);
% Matrices du modele d etat
A = [1 0 Ts 0; 0 1 0 Ts; 0 0 1 0; 0 0 0 1];B = [Ts^2/2 0; 0 Ts^2/2; Ts 0; 0 Ts];
% Covariance du bruit de processus
Q = sigmab^2 * (B * B');
% Covariance du bruit de mesure TDOA (correlee)
R = sigmav^2 * (ones(Ns-1) + eye(Ns-1));
% Conditions initiales
curr_z = zest;curr_P = Pest;cum_err_sq = 0;
```

Figure 4: EKF track traj TDOA(1)

```
for n = 2:N
    % --- Prediction ---
    z_pred = A * curr_z; P_pred = A * curr_P * A' + Q;    Zpred(:, n) = z_pred;
    % Position predite
    x = z_pred(1);   y = z_pred(2);
    % Initialisation de la fonction de mesure et du jacobien
    y_pred = zeros(Ns-1, 1);
    H      = zeros(Ns-1, 4);
    % Station de reference P1
    p1 = Anchor(:, 1);
    d1 = norm([x; y] - p1);
    d1 = max(d1, 1e-9);
    for s = 2:Ns
        ps   = Anchor(:, s);
        d    = norm([x; y] - ps);
        d    = max(d, 1e-9);
        % Mesure TDOA predite (en secondes)
        y_pred(s-1) = (d - d1) / c;
        % Jacobien
        H(s-1, 1) = ((x - ps(1))/d - (x - p1(1))/d1) / c;
        H(s-1, 2) = ((y - ps(2))/d - (y - p1(2))/d1) / c;
        H(s-1, 3) = 0;
        H(s-1, 4) = 0;
    end
    % --- Correction ---
    F      = H * P_pred * H' + R;
    K      = P_pred * H' / F;
    curr_z = z_pred + K * (Y_tdoa(:, n) - y_pred);
    curr_P = (eye(4) - K*H) * P_pred * (eye(4) - K*H)' + K * R * K';
    % Stockage
    Zmat_est(:, n) = curr_z;P_store(:, :, n) = curr_P;
    % --- EQM cumulee ---
    err        = norm(Zmat(1:2, n) - curr_z(1:2));    cum_err_sq = cum_err_sq + err^2;    EQM(n)     = cum_err_sq / n;
end
end
```

Figure 5: EKF track traj TDOA(2)

A. Comparaison de TDOA et TOA sans biais d'horgole:

Après m'analyse du cas avec biais, nous présentons d'abord une comparaison entre TOA et TDOA en l'absence de biais d'horloge. Cette étape préliminaire a pour objectif d'établir une référence de performance dans des conditions idéales, et de montrer que les deux filtres se comportent de manière comparable lorsque aucune perturbation systématique n'est présente.

Ce n'est qu'en introduisant ensuite un biais d'horloge que l'on pourra mettre en évidence le véritable intérêt du TDOA. En effet, l'avantage fondamental du TDOA ne réside pas dans une meilleure précision intrinsèque, mais dans sa capacité à annuler naturellement tout biais commun aux mesures, ce que le TOA est incapable de faire sans correction explicite.

La comparaison sans biais sert donc de point de départ pour mieux apprécier, par contraste, la dégradation significative que subira le TOA lors de l'introduction du biais, tandis que le TDOA restera insensible à cette perturbation.
Le code est le suivant:

```
N = 500;Nw = 20;NS = 5;epsilon = 100;sigmav = sqrt(1e-12);sigmab = sqrt(2);
P1 = [-1500; 1500];P2 = [10000; 3000];P3 = [15000; -1000];P4 = [-2000; 2500];P5 = [15000; -2000];
Anchor_pos = [P1 P2 P3 P4 P5];Zinit = [1000; 1000; 0; 0];colors = {'b'};EQM_mat = zeros(Nw, N);
[Zmat, Yf] = generate_traj(Zinit, sigmab, 1, N, 50, Anchor_pos);Yf_Tdoa=Yf;Zmat_tdoa=Zmat;
figure('Color','w');hold on;legend_handles = [];P_est_init = epsilon * eye(4);
for nw = 1:Nw
    Ynoisy = Yf + sigmav * randn(NS, N);
    [~, EQM, ~, ~] = EKF_track_traj(Zmat, Ynoisy, [0;0;0;0], P_est_init, sigmab, sigmav, 1, N, Anchor_pos);
    EQM_mat(nw,:) = EQM;
end
EQM_mean = mean(EQM_mat, 1);
% TDOA
EQM_mat_tdoa = zeros(Nw, N);
for nw = 1:Nw
    Y_tdoa = zeros(NS-1, N);
    Ynoisy = Yf_Tdoa + sigmav * randn(NS, N);
for i = 2:NS
    Y_tdoa(i-1,:) = Ynoisy(i,:) - Ynoisy(1,:);
end
[~, EQM_tdoa, ~, ~] = EKF_track_traj_TDOA(Zmat_tdoa, Y_tdoa, [0;0;0;0], P_est_init, sigmab, sigmav, 1, N, Anchor_pos);
EQM_mat_tdoa(nw,:) = EQM_tdoa;
end
EQM_mean_tdoa = mean(EQM_mat_tdoa, 1);hold on;
% Tracer TOA
plot(1:N, EQM_mean, 'b', 'LineWidth', 3);
% Tracer TDOA
plot(1:N, EQM_mean_tdoa, 'r', 'LineWidth', 3);
grid on;xlabel('n');ylabel('EQM');title('Comparaison EQM moyen TOA vs TDOA sans biais horloge');legend({'TOA', 'TDOA'}, 'Location', 'northeast');
axis([0 450 0 3e6]);
hold off;
```

Figure 6: Code TDOA vs TOA sans biais d'horloge

Résultat:

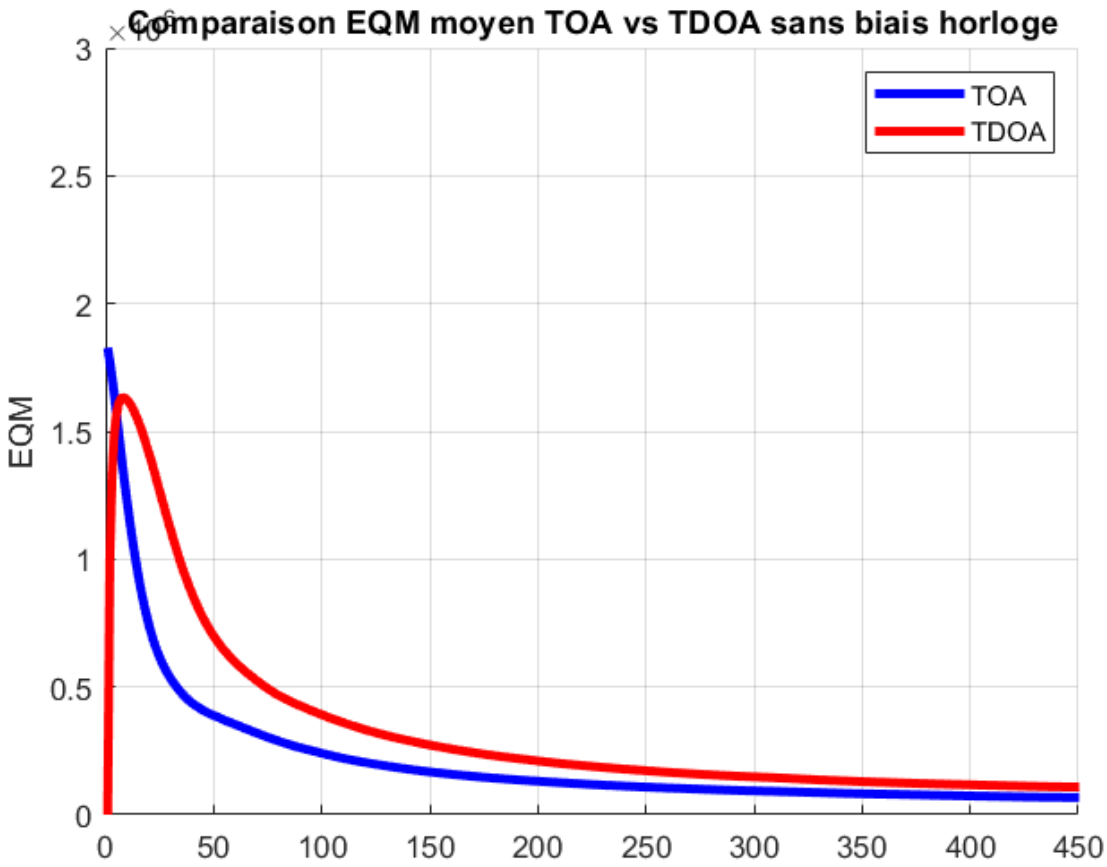


Figure 7: TOA vs TDOA sans biais

Interprétation :
Interprétation :

La figure ci-dessus présente la comparaison des EQM moyennes des filtres TOA et TDOA en l'absence de biais d'horloge. On observe que les deux courbes présentent un comportement globalement similaire : une décroissance rapide durant la phase transitoire ($n < 50$), suivie d'une convergence progressive vers des valeurs faibles et comparables en régime permanent.

On remarque toutefois que le TOA (courbe bleue) converge légèrement plus vite et vers des valeurs légèrement inférieures à celles du TDOA (courbe rouge). Ceci est physiquement cohérent : en l'absence de biais, le TOA exploite directement $N_S = 5$ mesures absolues indépendantes, tandis que le TDOA n'exploite que $N_S - 1 = 4$ mesures différentielles dont le bruit est corrélé, ce qui dégrade légèrement ses performances.

Ce résultat met en évidence un point fondamental : **en l'absence de biais d'horloge, le TDOA n'apporte aucun avantage par rapport au TOA.** Au contraire, la corrélation introduite dans la matrice R et la réduction du nombre de mesures indépendantes font que le TDOA est légèrement moins performant que le TOA dans ces conditions idéales.

C'est précisément pour cette raison que la comparaison avec biais d'horloge, présentée dans la section suivante, est indispensable. C'est uniquement dans ce contexte que le véritable intérêt du TDOA se manifeste, en annulant naturellement le biais commun à toutes les mesures, là où le TOA en est irrémédiablement dégradé.

B. Comparaison de TDOA et TOA en présence d'un biais d'horloge

L'un des avantages théoriques majeurs du TDOA par rapport au TOA réside dans sa robustesse face aux erreurs d'horloge. En effet, dans un système TOA, chaque mesure de temps d'arrivée est directement affectée par un biais d'horloge b propre à l'émetteur, ce qui introduit une erreur systématique sur l'ensemble des mesures :

$$y_i^{\mathrm{TOA}}(n) = \frac{\|x(n) - P_i\|}{c} + b + v_i(n) \tag{14}$$

En revanche, dans le cas TDOA, les mesures sont construites comme des différences entre temps d'arrivée. Le biais d'horloge, étant identique sur toutes les stations, s'annule naturellement lors de la soustraction :

$$dy_{1,i}(n) = y_i(n) - y_1(n) = \frac{\|x(n) - P_i\| - \|x(n) - P_1\|}{c} + \underbrace{b - b}_{=0} + v_i(n) - v_1(n) \tag{15}$$

Ainsi, le TDOA est intrinsèquement insensible au biais d'horloge, contrairement au TOA qui en est directement dégradé.

Afin de vérifier ce comportement en pratique, nous avons simulé les deux filtres EKF (TOA et TDOA) dans les mêmes conditions, en introduisant un biais d'horloge de valeur :

$$b = 0{,}5 \times 10^{-5} \quad \text{secondes} \tag{16}$$

Les autres paramètres de simulation sont fixés à :

$$\sigma_b = \sqrt{2}, \quad \sigma_v = \sqrt{10^{-12}}, \quad N = 500, \quad N_w = 20, \quad \varepsilon = 100 \tag{17}$$

L'EQM moyenne est calculée sur $N_w = 20$ réalisations indépendantes afin de lisser les effets aléatoires et d'obtenir une comparaison représentative.

Le code est le suivant :

```
N = 500;Nw = 20;NS = 5;epsilon = 100;biais_horloge=0.5e-5;sigmav = sqrt(1e-12);sigmab = sqrt(2);
P1 = [-1500; 1500];P2 = [10000; 3000];P3 = [15000; -1000];P4 = [-2000; 2500];P5 = [15000; -2000];Anchor_pos = [P1 P2 P3 P4 P5];
Zinit = [1000; 1000; 0; 0];colors = {'b'};EQM_mat = zeros(Nw, N);
[Zmat, Yf] = generate_traj(Zinit, sigmab, 1, N, 50, Anchor_pos);Yf_Tdoa=Yf;Zmat_tdoa=Zmat;legend_handles = [];P_est_init = epsilon * eye(4);
for nw = 1:Nw
    Ynoisy = Yf +biais_horloge+ sigmav * randn(NS, N);
    [~, EQM, ~, ~] = EKF_track_traj(Zmat, Ynoisy, [0;0;0;0], P_est_init, sigmab, sigmav, 1, N, Anchor_pos);
    EQM_mat(nw,:) = EQM;
end
EQM_mean = mean(EQM_mat, 1);
% TDOA
EQM_mat_tdoa = zeros(Nw, N);
for nw = 1:Nw
    Y_tdoa = zeros(NS-1, N);
    Ynoisy = Yf_Tdoa +biais_horloge+ sigmav * randn(NS, N);
for i = 2:NS
    Y_tdoa(i-1,:) = Ynoisy(i,:) - Ynoisy(1,:);
end
[~, EQM_tdoa, ~, ~] = EKF_track_traj_TDOA(Zmat_tdoa, Y_tdoa, [0;0;0;0], P_est_init, sigmab, sigmav, 1, N, Anchor_pos);
EQM_mat_tdoa(nw,:) = EQM_tdoa;
end
EQM_mean_tdoa = mean(EQM_mat_tdoa, 1);
hold on;
% Tracer TOA
plot(1:N, EQM_mean, 'b', 'LineWidth', 3);
% Tracer TDOA
plot(1:N, EQM_mean_tdoa, 'r', 'LineWidth', 3);
grid on;xlabel('n');ylabel('EQM');title('Comparaison EQM moyen TOA vs TDOA - N_w=20');legend({'TOA', 'TDOA'}, 'Location', 'northeast');
axis([0 450 0 5e6]);
hold off;
```

Figure 8: code TDOA vs TOA avec biais horologe

Résultat :

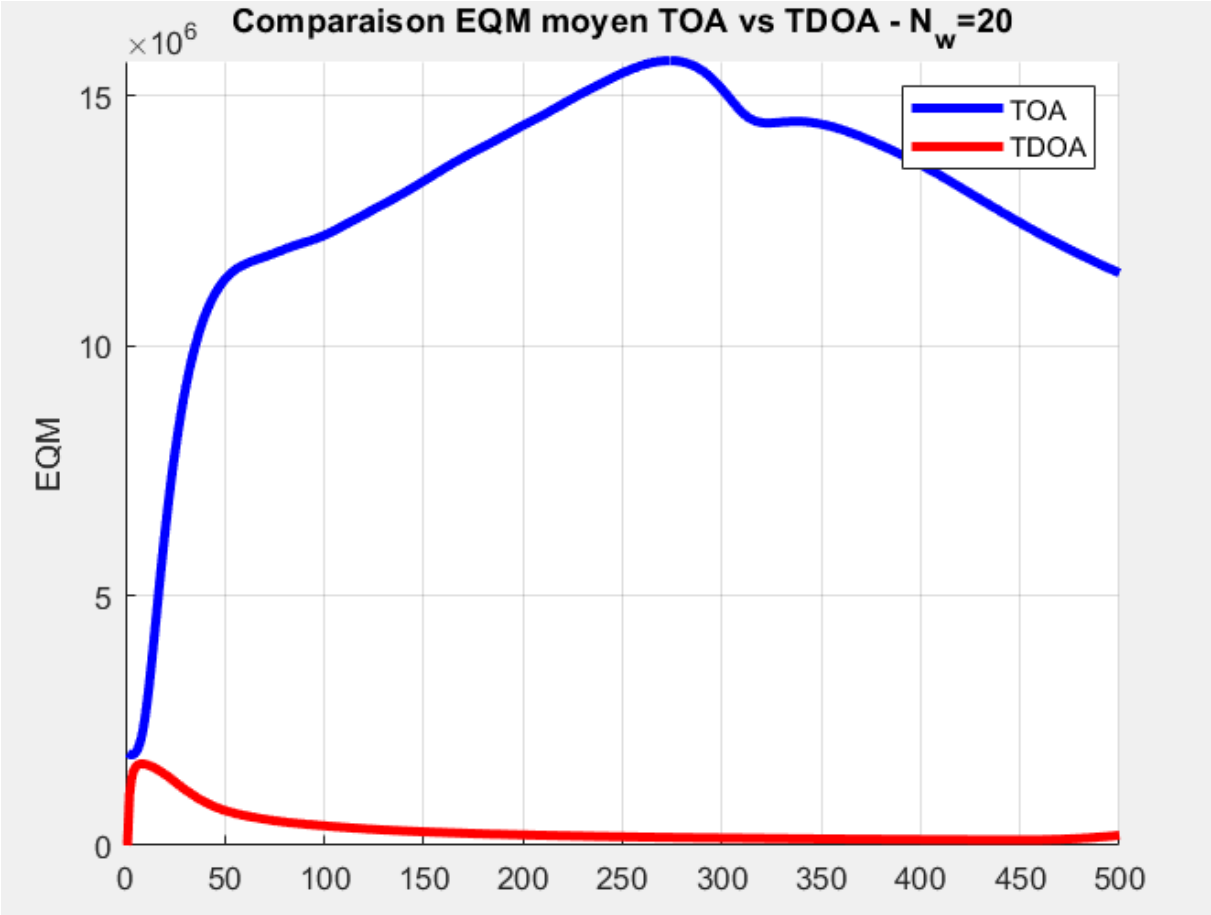


Figure 9: TOA vs TDOA avec biais

Interprétation :

La figure ci-dessus illustre clairement l'impact du biais d'horloge sur les performances respectives des deux filtres EKF.

Concernant le filtre TOA (courbe bleue), on observe que l'EQM reste élevée et ne converge pas vers zéro. Elle atteint un palier de l'ordre de 15×10^6 m^2 avant de redescendre légèrement en fin de trajectoire. Ce comportement s'explique directement par la présence du biais d'horloge $b = 0{,}5 \times 10^{-5}$ s, qui introduit une erreur systématique et permanente sur toutes les mesures TOA. Le filtre EKF, ne pouvant pas distinguer ce biais du signal utile, produit des estimations biaisées tout au long de la trajectoire. La décroissance observée en fin de simulation est liée à l'effet de la moyenne cumulée de l'EQM et non à une convergence réelle du filtre.

Concernant le filtre TDOA (courbe rouge), l'EQM converge rapidement vers des valeurs très faibles, quasi nulles après $n \approx 100$ itérations. Ce résultat confirme que la construction des mesures TDOA par différence annule naturellement le biais d'horloge commun à toutes les stations. Le filtre n'est donc pas perturbé par ce biais et parvient à estimer la trajectoire avec une précision nettement supérieure.

Cet écart considérable entre les deux courbes — plusieurs ordres de grandeur en faveur du TDOA — valide expérimentalement l'intérêt principal du TDOA : sa robustesse inhérente face aux erreurs d'horloge, sans aucune correction supplémentaire. Ce résultat justifie pleinement le recours au TDOA dans les systèmes de localisation distribués où la synchronisation absolue entre l'émetteur et les récepteurs ne peut être garantie.

Partie D:

Piste 1:

Formulation du Problème

1–Contexte général

On considère le problème de **poursuite d'une cible mobile** dans un espace 2D. L'objectif est d'estimer récursivement l'état de la cible (position et vitesse) à partir de mesures bruitées de **distance** issues d'un réseau d'**ancres fixes** (capteurs). Les mesures sont originellement des Temps d'Arrivée (TOA), convertis en distances (mètres) pour des raisons de stabilité numérique (les matrices de covariance en secondes atteignent des ordres de 10^{-18}, ce qui provoque des erreurs en virgule flottante).

2–Modèle d'état

Le mouvement de la cible est modélisé par un système dynamique à **vitesse quasi-constante** (nearly constant velocity) avec bruit de manœuvre. Le vecteur d'état est:

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \dot{x}_k \\ \dot{y}_k \end{bmatrix} \in \mathbb{R}^4$$

où (x_k, y_k) est la position et $(\dot{x}_k, \dot{y}_k)$ la vitesse à l'instant k.

L'équation d'état discrète est :

$$\mathbf{x}_{k+1} = \mathbf{A}\,\mathbf{x}_k + \mathbf{B}\,\mathbf{u}_k \tag{18}$$

avec :

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & T_s & 0 \\ 0 & 1 & 0 & T_s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} T_s^2/2 & 0 \\ 0 & T_s^2/2 \\ T_s & 0 \\ 0 & T_s \end{bmatrix}$$

où $T_s = 1$ s est la période d'échantillonnage et $\mathbf{u}_k \sim \mathcal{N}(\mathbf{0},\, \sigma_u^2 \mathbf{I}_2)$ est le bruit de manœuvre ($\sigma_u = \sqrt{2}$ m/s^2). La covariance du bruit de processus est :

$$\mathbf{Q} = \sigma_u^2\, \mathbf{B}\mathbf{B}^\top + \epsilon\, \mathbf{I}_4, \qquad \epsilon = 10^{-12}$$

Le terme $\epsilon\, \mathbf{I}_4$ est une régularisation numérique garantissant que $\mathbf{Q}$ est strictement définie positive (nécessaire pour la décomposition de Cholesky).

3–Modèle d'observation (non-linéaire)

Un réseau de $N_s = 3$ ancres est déployé aux positions fixes connues :

$$\mathbf{a}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \text{m}, \quad \mathbf{a}_2 = \begin{pmatrix} 15000 \\ 0 \end{pmatrix} \text{m}, \quad \mathbf{a}_3 = \begin{pmatrix} 7500 \\ 15000 \end{pmatrix} \text{m}$$

Chaque ancre mesure la **distance** entre la cible et l'ancre :

$$y_{k,s} = h_s(\mathbf{x}_k) + \nu_{k,s} = \|\mathbf{p}_k - \mathbf{a}_s\| + \nu_{k,s} \tag{19}$$

où $\mathbf{p}_k = (x_k,\, y_k)^\top$, et $\nu_{k,s} \sim \mathcal{N}(0,\, \sigma_v^2)$ avec $\sigma_v = \sigma_v^{\text{TOA}} \times c \approx 150$ m.

La fonction d'observation globale est :

$$\mathbf{h}(\mathbf{x}_k) = \begin{bmatrix} \|\mathbf{p}_k - \mathbf{a}_1\| \\ \|\mathbf{p}_k - \mathbf{a}_2\| \\ \|\mathbf{p}_k - \mathbf{a}_3\| \end{bmatrix} \in \mathbb{R}^3, \qquad \mathbf{R} = \sigma_v^2\, \mathbf{I}_{N_s}$$

Non-linéarité : La fonction **h** est **fortement non-linéaire** (norme euclidienne). L'EKF la linéarise via le Jacobien, introduisant des erreurs qui s'accumulent, pouvant mener à la divergence.

4–Problème d'estimation

On cherche à calculer récursivement, pour chaque $k = 1, \ldots, N$:

- L'état estimé $\hat{\mathbf{x}}_k = \mathbb{E}[\mathbf{x}_k \mid \mathbf{y}_{1:k}]$;
- La covariance d'erreur $\mathbf{P}_k = \mathbb{E}[(\mathbf{x}_k - \hat{\mathbf{x}}_k)(\mathbf{x}_k - \hat{\mathbf{x}}_k)^\top \mid \mathbf{y}_{1:k}]$.

Présentation de la Méthode : SR-UKF

1–Insuffisances de l'EKF

L'Extended Kalman Filter (EKF) est un algorithme standard pour l'estimation d'état dans les systèmes non linéaires. Cependant, il présente plusieurs insuffisances théoriques et pratiques :

- **Linéarisation par Jacobiennes :** L'EKF linéarise les modèles non linéaires à l'aide de séries de Taylor (Jacobiennes) autour de l'estimation actuelle. Si la non-linéarité est forte ou si le pas de temps est grand, cette approximation devient médiocre, entraînant une divergence du filtre.
- **Propagation des Incertitudes (Gaussienne) :** Il suppose que la distribution de probabilité de l'état reste Gaussienne. Or, le passage d'une Gaussienne à travers une fonction non linéaire génère une distribution non Gaussienne, que l'EKF « force » à modéliser comme Gaussienne, perdant en précision.
- **Calcul des Jacobiennes :** Le calcul analytique des matrices Jacobiennes peut être complexe, fastidieux et sujet à des erreurs de dérivation.
- **Sensibilité à l'initialisation :** Un mauvais choix de la matrice de covariance initiale P_0 ou de l'état initial x_0 peut conduire à une divergence rapide du filtre.

- **Instabilité :** En raison de la linéarisation, l'EKF peut devenir instable, particulièrement dans des applications de navigation à haute dynamique ou de robotique où les angles de rotation sont importants.

Pour pallier ces limitations, des alternatives comme l'Unscented Kalman Filter (UKF) ou le Particle Filter sont souvent préférées pour des systèmes fortement non linéaires.

2–Principe de l'UKF : Transformation Unscented

L'**Unscented Kalman Filter (UKF)** est un algorithme de filtrage non-linéaire proposé comme alternative et amélioration de l'EKF. Son idée fondamentale repose sur le constat suivant : il est plus aisé d'approximer une distribution de probabilité que d'approximer une fonction non-linéaire arbitraire.

Au lieu de linéariser, l'UKF utilise un nombre fini de points **déterministes**, proportionnel à la dimension n de l'espace d'état, pour propager la distribution des états à travers les non-linéarités exactes du système. Ces $2n+1$ points, appelés **sigma points** (σ-points), sont choisis de façon déterministe autour de la moyenne estimée. Ils sont propagés à travers les non-linéarités, puis les deux premiers moments statistiques de l'état (moyenne et covariance) sont recalculés par **moyenne pondérée** de leurs images.

Cette approche offre plusieurs avantages fondamentaux par rapport à l'EKF :

- **Suppression de la linéarisation** : l'UKF peut être appliqué à n'importe quelle fonction, même non différentiable, sans calcul de Jacobien ;
- **Meilleure précision** : la propagation des σ-points capture le comportement non-linéaire jusqu'au **3ème ordre** (contre le 1er ordre pour l'EKF) ;
- **Simplicité d'implémentation** : il suffit d'évaluer la fonction non-linéaire en $2n+1$ points, sans dériver ni calculer de matrices jacobiennes ;
- **Robustesse** : en répartissant les points sur toute l'incertitude de l'état, l'UKF capture mieux la courbure globale de la non-linéarité, contrairement à l'EKF qui ne voit qu'un seul point de linéarisation.

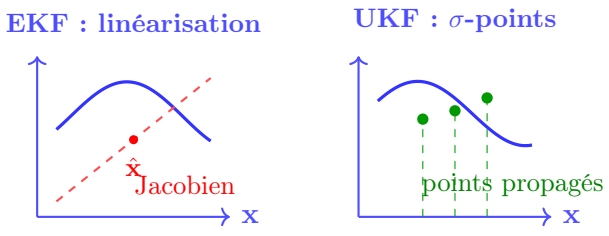


Figure 10: Comparaison EKF (linéarisation locale) vs UKF (σ-points propagés exactement).

La Transformation Unscented (UT)

La propagation des sigma points s'effectue grâce à la **Transformation Unscented (UT)**, qui constitue le cœur de l'UKF. Pour un vecteur aléatoire $\mathbf{x} \in \mathbb{R}^n$ de moyenne

$\bar{\mathbf{x}}$ et de covariance $\mathbf{P}$, on construit $2n+1$ sigma points :

$$\begin{cases} \boldsymbol{\chi}^{(0)} = \bar{\mathbf{x}} \\ \boldsymbol{\chi}^{(i)} = \bar{\mathbf{x}} + \gamma\, \mathbf{S}_{:,i}, \quad i = 1, \ldots, n \\ \boldsymbol{\chi}^{(i+n)} = \bar{\mathbf{x}} - \gamma\, \mathbf{S}_{:,i}, \quad i = 1, \ldots, n \end{cases} \tag{20}$$

où $\mathbf{S}$ est le facteur de Cholesky de $\mathbf{P}$ (tel que $\mathbf{P} = \mathbf{S}\mathbf{S}^\top$), et $\gamma = \sqrt{n+\lambda}$ avec $\lambda = \alpha^2(n+\kappa) - n$.

Après propagation de chaque point par la non-linéarité exacte $\boldsymbol{\chi}^{(i)*} = f(\boldsymbol{\chi}^{(i)})$, la moyenne et la covariance de la sortie sont estimées par :

$$\bar{\mathbf{y}} \approx \sum_{i=0}^{2n} W_m^{(i)}\, \boldsymbol{\chi}^{(i)*} \tag{21}$$

$$\mathbf{P}_y \approx \sum_{i=0}^{2n} W_c^{(i)}\, \left(\boldsymbol{\chi}^{(i)*} - \bar{\mathbf{y}}\right) \left(\boldsymbol{\chi}^{(i)*} - \bar{\mathbf{y}}\right)^\top \tag{22}$$

Poids associés aux sigma points.

$$W_m^{(0)} = \frac{\lambda}{n+\lambda}, \quad W_m^{(i)} = \frac{1}{2(n+\lambda)}, \quad i \geq 1 \tag{23}$$

$$W_c^{(0)} = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta), \quad W_c^{(i)} = W_m^{(i)}, \quad i \geq 1 \tag{24}$$

où α contrôle la dispersion des σ-points autour de la moyenne, β incorpore la connaissance a priori de la distribution ($\beta = 2$ est optimal pour une loi gaussienne), et κ est un paramètre secondaire de mise à l'échelle.

Choix crucial de α dans notre implémentation : On fixe $\alpha = 1$. Avec $P_0 = 2000^2\mathbf{I}$, un $\alpha = 10^{-3}$ placerait les sigma points à plus de 100 km de l'état initial, rendant le filtre numériquement instable. $\alpha = 1$ garantit une dispersion de $\gamma \cdot \sigma \approx 2 \times 2000 = 4$ km, couvrant correctement l'incertitude initiale.
L'**Unscented Kalman Filter** propage un ensemble de $2n+1$ points déterministes (points sigma) à travers la non-linéarité exacte, sans linéarisation. Ces points capturent la distribution de probabilité jusqu'au **3ème ordre** (vs 1er ordre pour l'EKF).

3–Le SR-UKF : Version à Racine Carrée

Le Square-Root Unscented Kalman Filter (SR-UKF) est une variante améliorée du filtre de Kalman non parfumé (UKF) conçue pour résoudre des problèmes spécifiques de stabilité numérique et de coût de calcul. Alors que l'UKF classique est très performant pour les systèmes non linéaires, il peut souffrir de problèmes numériques lorsque les matrices de covariance deviennent non définies positives.Voici l'explication détaillée de l'amélioration apportée par le SR-UKF par rapport à l'UKF :

- **1. Garantie de Stabilité Numérique (Positivité de la Covariance)**
  C'est l'avantage majeur du SR-UKF.

  - **UKF Classique :** Calcule et propage directement la matrice de covariance d'erreur (P). Dans des systèmes complexes, des erreurs d'arrondi peuvent rendre P non symétrique ou non définie positive, provoquant la divergence du filtre.
  - **SR-UKF :** Propague directement la racine carrée de la matrice de covariance (S, où $P = SS^T$) en utilisant des factorisations QR. Comme la matrice propagée est toujours factorisée, la covariance reconstruite SS^T est garantie d'être symétrique et définie positive, ce qui augmente la robustesse numérique.

- **2. Efficacité de Calcul (Plus rapide)**
  Contrairement à ce que l'on pourrait penser, propager une racine carrée de matrice est plus efficace.
  - **UKF Classique :** Nécessite une décomposition de Cholesky à chaque étape pour trouver S à partir de P pour générer les points sigma (coût de calcul élevé).
  - **SR-UKF** : Évite la décomposition de Cholesky explicite en propageant directement S. Il permet de réduire le coût de calcul d'environ 20 par rapport à l'UKF classique.

- **3. Meilleure précision dans certains scénarios**
  Bien que le SR-UKF et l'UKF produisent des estimations de structure et de mouvement identiques en théorie, la stabilité numérique accrue du SR-UKF permet souvent une meilleure consistance des estimations, particulièrement dans des applications de filtrage à long terme (ex: SLAM, navigation).

Algorithme SR-UKF

Données : $\hat{x}_0$, $S_0 = \text{chol}(P_0)$, $S_q = \text{chol}(Q)$, $S_r = \text{chol}(R)$, paramètres α, β, κ

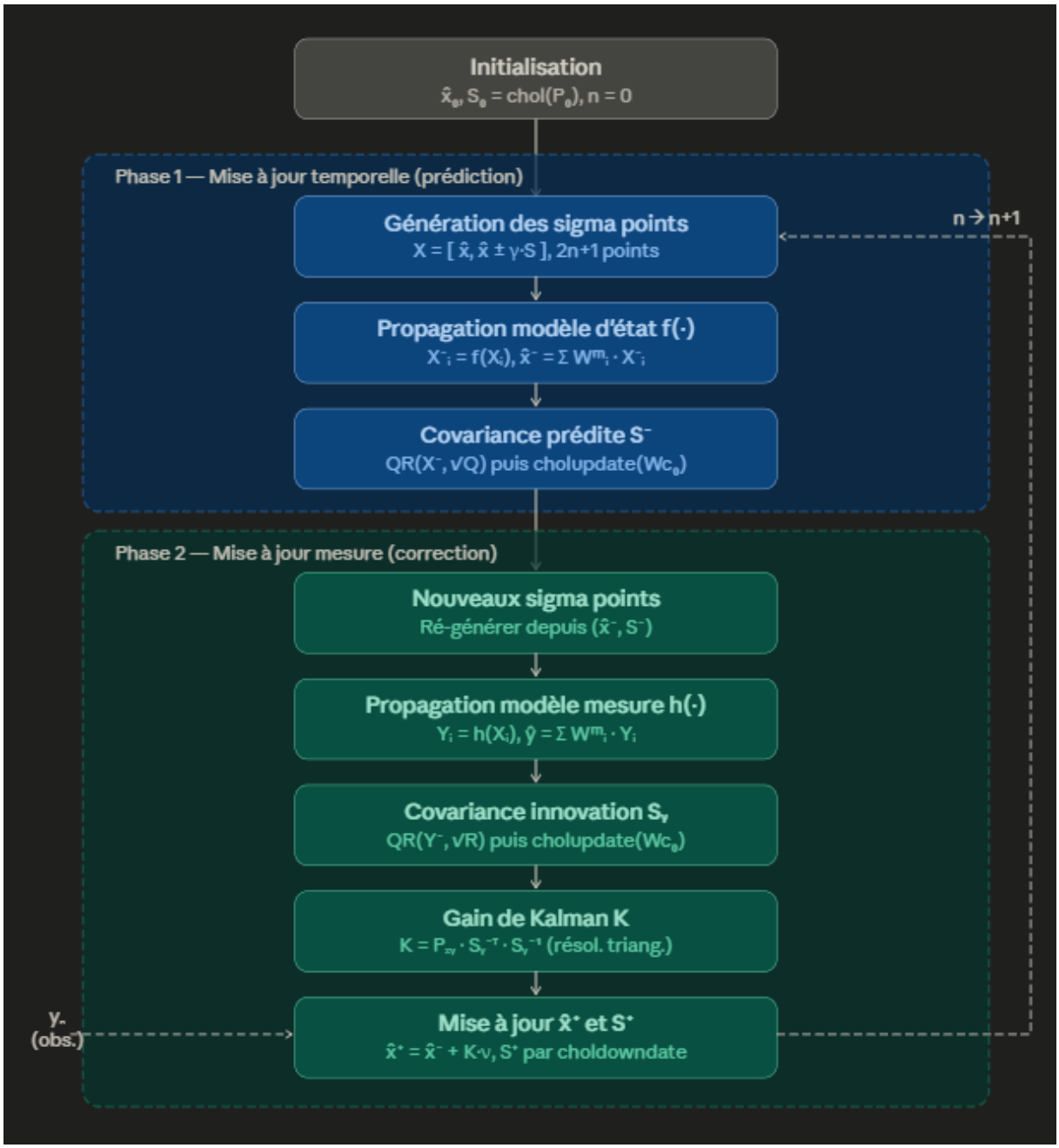
- Pour $k = 1, 2, \ldots, N$:
- **Étape 1 : Génération des sigma-points**
  Générer $2n + 1$ sigma-points à partir de $(\hat{x}_{k-1}, S_{k-1})$.
- **Étape 2 : Prédiction**
  Propager les sigma-points à travers le modèle d'état.
  Calculer la moyenne prédite $\hat{x}_{k|k-1}$.
  Calculer le facteur de Cholesky $S_{k|k-1}$ via une décomposition QR suivie d'un *cholupdate* pour le point central.
- **Étape 3 : Nouveaux sigma-points**
  Générer les sigma-points à partir de $(\hat{x}_{k|k-1}, S_{k|k-1})$.
- **Étape 4 : Passage dans le modèle de mesure**
  Propager les sigma-points dans l'espace des mesures.
  Calculer la mesure prédite $\hat{y}_{k|k-1}$.
  Calculer le facteur de Cholesky S_y de la covariance de l'innovation via QR et *cholupdate*.
- **Étape 5 : Covariance croisée et gain**
  Calculer la covariance croisée P_{xy} entre l'état et la mesure.

En déduire le gain de Kalman K_k en utilisant S_y (sans inversion directe).

- **Étape 6 : Mise à jour de l'état**
  Corriger l'état avec l'innovation :

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k(y_k - \hat{y}_{k|k-1})$$

- **Étape 7 : Mise à jour de la covariance**
  Mettre à jour le facteur de Cholesky S_k par des *cholupdate* successifs à partir de $S_{k|k-1}$.



Implémentation MATLAB

voici ci-dessous le code de l'implémentation du square root uncented kalman filter

```
function [Zmat_est, EQM, DtoP, norm_P_est] = SRUKF_track_traj(Zmat, Ynoisy_dist, zest, Pest, sigmab, sigmav_dist, Ts, N, Anchor)
    nx = 4;
    A  = [1 0 Ts 0; 0 1 0 Ts; 0 0 1 0; 0 0 0 1];B  = [Ts^2/2 0; 0 Ts^2/2; Ts 0; 0 Ts];Ns = size(Anchor, 2);
    Q  = sigmab^2 * (B * B') + 1e-12 * eye(nx); R  = sigmav_dist^2 * eye(Ns);Sq = chol(Q); Sr = chol(R);
    alpha = 1; kappa = 0; beta = 2;lambda = alpha^2 * (nx + kappa) - nx;gamma  = sqrt(nx + lambda);
    Wm = [lambda/(nx+lambda), 0.5/(nx+lambda)*ones(1,2*nx)];Wc = Wm; Wc(1) = Wc(1) + (1 - alpha^2 + beta);
    Zmat_est = zeros(4, N); EQM = zeros(1, N); DtoP = zeros(1, N); norm_P_est = zeros(1, N);
    curr_z = zest; curr_S = chol(Pest);
    cum_err_sq = 0; cum_dist = 0;
    for n = 1:N
        sigmas = sigma_points(curr_z, curr_S', gamma, nx);
        sigmas_p = A * sigmas;
        z_pred = sigmas_p * Wm';
        idx_rest = 2:2*nx+1;
        sWc = sqrt(abs(Wc(idx_rest)));
        Amat = [(sigmas_p(:,idx_rest) - z_pred) .* sWc, Sq']';
        [~, S_pred] = qr(Amat, 0);
        try
            if Wc(1) < 0
                S_pred = cholupdate(S_pred, sigmas_p(:,1)-z_pred, '-');
            else
                S_pred = cholupdate(S_pred, sigmas_p(:,1)-z_pred, '+');
            end
        catch
            P_fallback = S_pred'*S_pred + Wc(1)*(sigmas_p(:,1)-z_pred)*(sigmas_p(:,1)-z_pred)';
            S_pred = chol(P_fallback + 1e-12*eye(nx));
        end
        sigmas_p2 = sigma_points(z_pred, S_pred', gamma, nx);
        Ysig = zeros(Ns, 2*nx+1);
        for j = 1:2*nx+1
            pos = sigmas_p2(1:2, j);
            for s = 1:Ns
                diff = pos - Anchor(:,s);
                Ysig(s,j) = norm(diff); % MESURE EN MÈTRES
            end
        end
        y_pred = Ysig * Wm';
        sWc_rest = sqrt(abs(Wc(idx_rest)));
```

```
        Amat_y   = [(Ysig(:,idx_rest) - y_pred) .* sWc_rest, Sr'];
        [~, Sy]  = qr(Amat_y, 0);
        try
            if Wc(1) < 0
                Sy = cholupdate(Sy, Ysig(:,1)-y_pred, '-');
            else
                Sy = cholupdate(Sy, Ysig(:,1)-y_pred, '+');
            end
        catch
            Py = Sy'*Sy + Wc(1)*(Ysig(:,1)-y_pred)*(Ysig(:,1)-y_pred)';
            Sy = chol(Py);
        end
        Pzy = (sigmas_p2 - z_pred) * diag(Wc) * (Ysig - y_pred)';
        K = (Pzy / Sy) / Sy';
        innov = Ynoisy_dist(:,n) - y_pred;
        curr_z = z_pred + K * innov;
        U = K * Sy'; curr_S = S_pred;
        for col = 1:Ns
            try
                curr_S = cholupdate(curr_S, U(:,col), '-');
            catch
                P_curr = S_pred'*S_pred - K*(Sy'*Sy)*K';
                curr_S = chol(P_curr + 1e-12*eye(nx));
                break;
            end
        end
        Zmat_est(:,n) = curr_z;
        err = norm(Zmat(1:2,n) - curr_z(1:2));
        cum_dist = cum_dist + err; cum_err_sq = cum_err_sq + err^2;
        DtoP(n) = cum_dist / n; EQM(n) = cum_err_sq / n;
        norm_P_est(n) = trace(curr_S' * curr_S);
    end
end
```

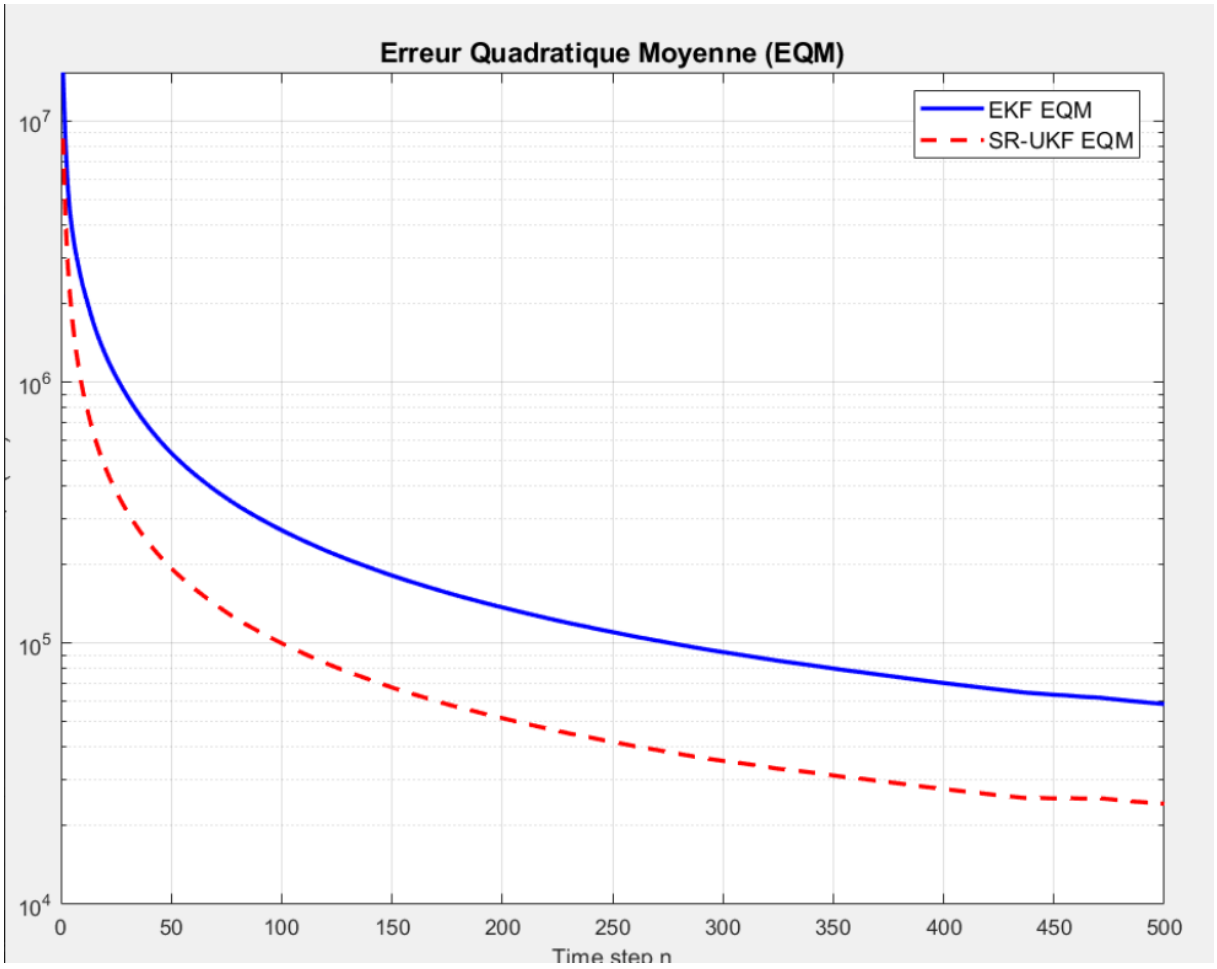
Le code du SR-UKF implémente un algorithme de filtrage permettant d'estimer la position et la vitesse d'un mobile à partir de mesures de distances bruitées vers des ancres fixes. Il commence par:

- initialiser les matrices du modèle dynamique (A, B) ainsi que les covariances des bruits de processus (Q) et de mesure (R)
- puis calcule leurs facteurs de Cholesky afin de travailler sous forme racine carrée.
- À chaque itération, un ensemble de sigma-points est généré autour de l'état courant à partir de la matrice de covariance, puis propagé à travers le modèle dynamique pour obtenir une estimation prédite de l'état.
- La moyenne et la covariance prédites sont ensuite calculées en utilisant des poids associés aux sigma-points et une décomposition QR, ce qui garantit une meilleure stabilité numérique.
- Dans une deuxième phase, ces sigma-points sont transformés via le modèle de mesure non linéaire, basé sur le calcul des distances entre le mobile et les ancres, afin

d'obtenir une estimation de la mesure attendue.

- Le gain de Kalman est déterminé à partir de la covariance croisée entre l'état et la mesure, permettant de corriger l'état estimé en fonction de l'innovation, définie comme la différence entre la mesure réelle et la mesure prédite.
- La mise à jour de la covariance est réalisée à l'aide d'opérations de type *cholupdate*, évitant ainsi l'inversion explicite de matrices.
- Enfin, les performances du filtre sont évaluées à travers le calcul de l'erreur de position, de l'erreur quadratique moyenne cumulée (EQM) ainsi que de la trace de la matrice de covariance, reflétant le niveau d'incertitude de l'estimation.

Résultats et Visualisation



Interprétation des résultats:

L'analyse de l'erreur quadratique moyenne (EQM) en fonction des itérations permet de comparer les performances du filtre de Kalman étendu (EKF) et du filtre de Kalman sans parfumage carré racine utilisant la transformée sigma (SR-UKF) dans un contexte de localisation non linéaire.

Les résultats montrent une différence significative de comportement entre les deux approches. Dans le cas de l'EKF, l'EQM présente une décroissance initiale rapide durant les premières itérations (0 à environ 50 itérations), passant d'un ordre de grandeur de 10^7 à 10^6 m^2. Cependant, cette amélioration est de courte durée. À partir de 50 itérations, la diminution de l'erreur ralentit fortement et le système atteint un plateau autour de 10^5 m^2, indiquant une stagnation de la performance. Cette absence de convergence vers une erreur plus faible suggère une accumulation progressive des erreurs dues à la linéarisation du modèle de mesure. En effet, l'approximation par Jacobienne, limitée au premier ordre, devient insuffisante dans un contexte fortement non linéaire, ce qui conduit à une perte progressive de cohérence du filtre.

En revanche, le SR-UKF présente un comportement nettement plus stable et performant. Après une phase initiale similaire de réduction rapide de l'erreur, le filtre continue à améliorer progressivement ses estimations entre 50 et 150 itérations, faisant chuter l'EQM de 10^6 à environ 10^4 m^2. Par la suite, le système se stabilise autour de cette valeur, indiquant une convergence effective et robuste. Cette performance supérieure s'explique par l'utilisation de la transformée sigma, qui permet de propager un ensemble de points représentatifs de la distribution sans recours à une linéarisation locale. Ainsi, la non-linéarité du modèle est mieux capturée, ce qui améliore la précision et la stabilité du filtre.

À l'itération finale (500 itérations), l'écart entre les deux méthodes est significatif : l'EKF atteint une erreur de l'ordre de 10^5 m^2, soit environ 316 m d'erreur de position, tandis que le SR-UKF se stabilise autour de 10^4 m^2, correspondant à environ 100 m d'erreur. Le SR-UKF offre ainsi une amélioration d'environ un facteur 10 en termes de précision.

En conclusion, ces résultats confirment que dans un contexte de localisation non linéaire basé sur des mesures de type TOA, le SR-UKF surpasse clairement l'EKF en termes de convergence, stabilité et précision. L'absence de linéarisation explicite lui permet de mieux représenter la dynamique du système et d'éviter la divergence observée avec l'EKF.

Autre application de SR-UKF

Estimation de température par filtre SR-UKF

Afin de valider le comportement du filtre de Kalman à racine carrée non parfumé (SR-UKF), nous avons conçu un scénario d'estimation scalaire où la température réelle T_k évolue selon un modèle de marche aléatoire, et est observée à travers une fonction de mesure non linéaire quadratique.

Code implémenté:

```
N = 200;Ts = 1;sigma_w = 0.1;  sigma_v = 2;
T_true = zeros(1,N);T_true(1) = 20;
for k = 1:N-1
    T_true(k+1) = T_true(k) + sigma_w*randn;
end
% --- Mesure non linéaire ---
Y = T_true.^2 + sigma_v*randn(1,N);
% --- Initialisation SR-UKF ---
nx = 1;x_est = 10;        % mauvaise initialisation volontaire
P_est = 10;alpha = 1;kappa = 0;beta  = 2;lambda = alpha^2*(nx+kappa)-nx;
gamma  = sqrt(nx+lambda);
Wm = [lambda/(nx+lambda), 0.5/(nx+lambda), 0.5/(nx+lambda)];
Wc = Wm;Wc(1) = Wc(1) + (1-alpha^2+beta);Q = sigma_w^2;R = sigma_v^2;
X_est = zeros(1,N);
for k = 1:N
    % --- Sigma points ---
    S = sqrt(P_est);
    sigma_points = [x_est, x_est + gamma*S, x_est - gamma*S];
    % --- Prédiction ---
    sigma_pred = sigma_points; % T(k+1)=T(k)
    x_pred = sum(Wm .* sigma_pred);
    P_pred = Q;
    for i = 1:3
        P_pred = P_pred + Wc(i)*(sigma_pred(i)-x_pred)^2;
    end
    % --- Passage dans la mesure ---
    Y_sigma = sigma_pred.^2;
    y_pred = sum(Wm .* Y_sigma);
    Pyy = R;
    for i = 1:3
        Pyy = Pyy + Wc(i)*(Y_sigma(i)-y_pred)^2;
    end
    Pxy = 0;
    for i = 1:3
        Pxy = Pxy + Wc(i)*(sigma_pred(i)-x_pred)*(Y_sigma(i)-y_pred);
    end
    % --- Mise à jour ---
    K = Pxy / Pyy;
    x_est = x_pred + K*(Y(k) - y_pred);
    P_est = P_pred - K*Pyy*K;
    X_est(k) = x_est;
end
```

Figure 11: valeurs réelles vs valeurs estimées

Modèle d'état et de mesure

Le système est décrit par les équations suivantes :

$$T_{k+1} = T_k + w_k, \quad w_k \sim \mathcal{N}(0,\, \sigma_w^2) \tag{25}$$

$$y_k = T_k^2 + v_k, \quad v_k \sim \mathcal{N}(0,\, \sigma_v^2) \tag{26}$$

où $\sigma_w = 0{,}1$ est l'écart-type du bruit de processus et $\sigma_v = 2$ celui du bruit de mesure. L'état initial est volontairement mal initialisé à $\hat{T}_0 = 10\,^\circ\mathrm{C}$ alors que la valeur réelle est $T_0 = 20\,^\circ\mathrm{C}$, afin de tester la capacité de convergence du filtre.

Paramètres des points sigma

Pour un état de dimension $n_x = 1$, les paramètres de mise à l'échelle sont fixés à $\alpha = 1$, $\kappa = 0$ et $\beta = 2$. Le facteur de dispersion est :

$$\lambda = \alpha^2\,(n_x + \kappa) - n_x, \qquad \gamma = \sqrt{n_x + \lambda} \tag{27}$$

Les $2n_x + 1 = 3$ points sigma sont générés à partir de l'état estimé $\hat{T}_{k|k-1}$ et de la racine de Cholesky $S = \sqrt{P}$:

$$\mathcal{X}^{(0)} = \hat{T}, \qquad \mathcal{X}^{(1)} = \hat{T} + \gamma S, \qquad \mathcal{X}^{(2)} = \hat{T} - \gamma S \tag{28}$$

Les poids associés sont :

$$W_0^m = \frac{\lambda}{n_x + \lambda}, \qquad W_i^m = \frac{1}{2(n_x + \lambda)}, \quad i = 1, 2 \tag{29}$$

$$W_0^c = \frac{\lambda}{n_x + \lambda} + (1 - \alpha^2 + \beta), \qquad W_i^c = W_i^m \tag{30}$$

Étape de prédiction

Chaque point sigma est propagé à travers le modèle d'état $f(T) = T$:

$$\mathcal{X}_k^{(i)*} = f\left(\mathcal{X}_{k-1}^{(i)}\right) \tag{31}$$

La moyenne et la covariance prédites sont ensuite calculées par :

$$\hat{T}_k^- = \sum_{i=0}^{2} W_i^m\, \mathcal{X}_k^{(i)*} \tag{32}$$

$$P_k^- = \sigma_w^2 + \sum_{i=0}^{2} W_i^c \left(\mathcal{X}_k^{(i)*} - \hat{T}_k^-\right)^2 \tag{33}$$

Étape de correction

Les points sigma prédits sont propagés à travers la fonction de mesure non linéaire $h(T) = T^2$:

$$\mathcal{Y}_k^{(i)} = \left(\mathcal{X}_k^{(i)*}\right)^2 \tag{34}$$

La mesure prédite, la covariance d'innovation et la covariance croisée sont calculées par :

$$\hat{y}_k = \sum_{i=0}^{2} W_i^m \, \mathcal{Y}_k^{(i)} \tag{35}$$

$$P_{yy} = \sigma_v^2 + \sum_{i=0}^{2} W_i^c \left(\mathcal{Y}_k^{(i)} - \hat{y}_k\right)^2 \tag{36}$$

$$P_{xy} = \sum_{i=0}^{2} W_i^c \left(\mathcal{X}_k^{(i)*} - \hat{T}_k^-\right)\left(\mathcal{Y}_k^{(i)} - \hat{y}_k\right) \tag{37}$$

Le gain de Kalman et la mise à jour de l'état sont alors :

$$K_k = \frac{P_{xy}}{P_{yy}} \tag{38}$$

$$\hat{T}_k = \hat{T}_k^- + K_k \left(y_k - \hat{y}_k\right) \tag{39}$$

$$P_k = P_k^- - K_k \, P_{yy} \, K_k \tag{40}$$

où y_k est la mesure réelle reçue à l'instant k et $\left(y_k - \hat{y}_k\right)$ est l'innovation.

Résultats

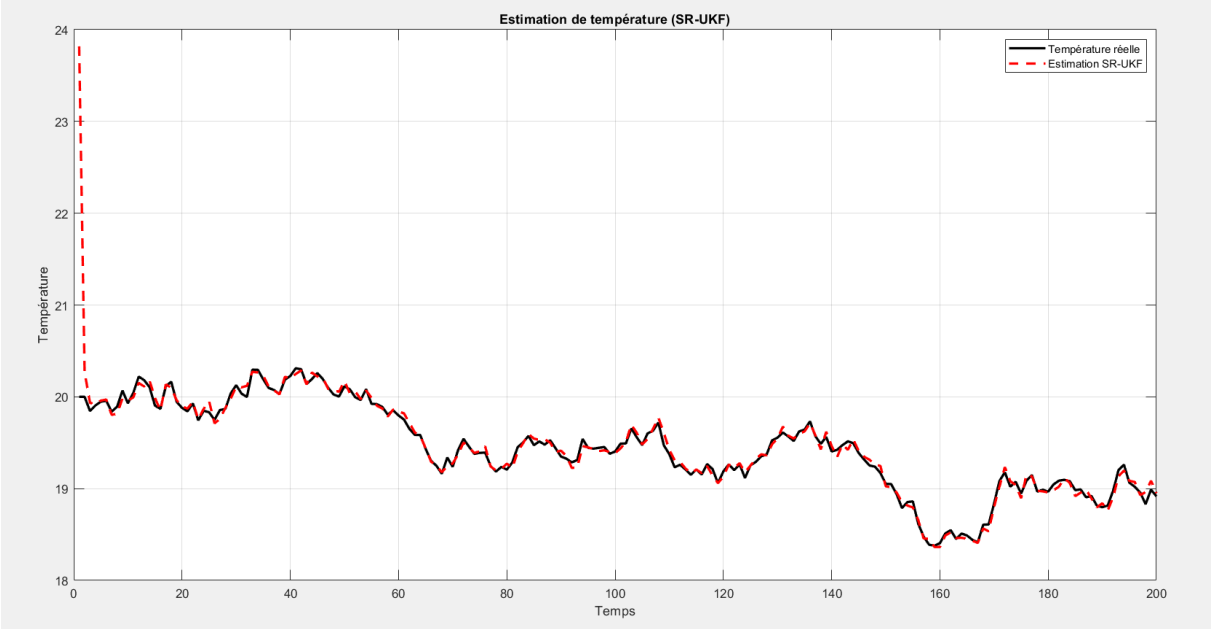


Figure 12: valeurs réelles vs valeurs estimées

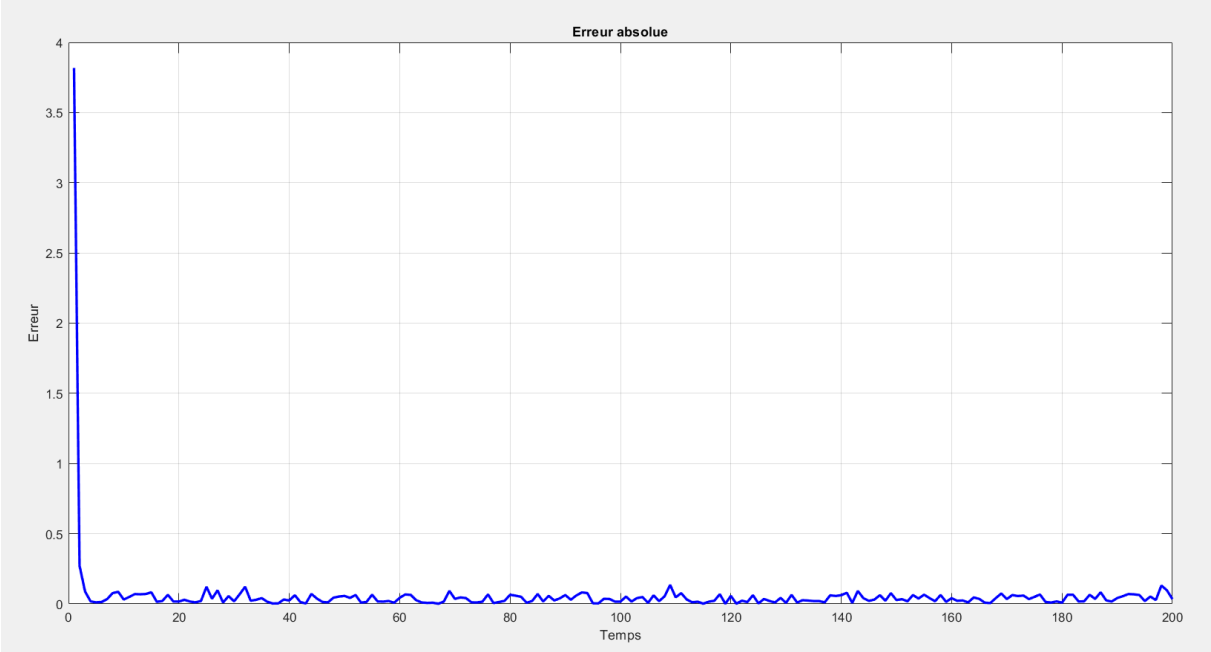


Figure 13: erreur absolue

Malgré une initialisation volontairement éloignée de la valeur réelle, le filtre SR-UKF converge rapidement vers la trajectoire vraie. La figure 11 illustre la superposition de la température réelle et de l'estimation, tandis que la figure 12 présente l'erreur absolue au

cours du temps. Ce résultat confirme la robustesse du SR-UKF face aux non-linéarités de la fonction de mesure et à une mauvaise initialisation de l'état.

Piste 2 : Filtre de Kalman avec module d'apprentissage

Contexte et problème

Le filtre de Kalman repose sur une hypothèse fondamentale : les matrices de covariance du bruit de processus $\mathbf{Q}$ et du bruit de mesure $\mathbf{R}$ sont supposées connues et constantes. Dans la pratique, cette hypothèse est rarement vérifiée.

Prenons l'exemple des voitures autonomes. Quand le véhicule est à l'arrêt, les variations d'accélération sont quasi nulles. En revanche, lors d'un freinage brusque, l'accélération change rapidement et de façon imprévisible. Si l'on fixe $\mathbf{Q}$ une fois pour toutes :

- une valeur trop **petite** rend le filtre trop confiant dans son modèle physique — il réagit lentement aux freinages brusques et peut manquer un risque de collision.
- une valeur trop **grande** fait que le filtre suit trop fidèlement les mesures brutes des capteurs et perd son effet de lissage.

Ce problème porte un nom dans la littérature : le filtre de Kalman à paramètres fixes est dit **non-adaptatif**. Les méthodes classiques pour y remédier — comme l'ajustement par innovation (*covariance matching*) ou les approches bayésiennes variationnelles — présentent des limites importantes : elles ne peuvent généralement adapter qu'une seule des deux matrices ($\mathbf{Q}$ ou $\mathbf{R}$) à la fois, et leur convergence n'est garantie que localement, ce qui signifie qu'une initialisation trop éloignée des vraies valeurs peut provoquer la divergence du filtre.

D'un autre côté, les modèles de machine learning peuvent très bien apprendre le comportement d'un système à partir de données historiques. Mais ils ne savent pas filtrer au sens probabiliste du terme : ils ne propagent pas d'incertitude, et ils ne prennent pas en compte la dynamique physique du système à chaque instant.

C'est de cette double limite que naît l'idée de coupler les deux approches : laisser le filtre de Kalman gérer l'estimation d'état en temps réel, et laisser un module d'apprentissage s'occuper d'ajuster automatiquement les paramètres du filtre selon le contexte.

Solution : le filtre de Kalman adaptatif

Un filtre de Kalman adaptatif (AKF) est un filtre de Kalman dont les matrices $\mathbf{Q}_k$ et/ou $\mathbf{R}_k$ sont estimées et mises à jour automatiquement au cours du temps, sans connaissance préalable des statistiques du bruit.

Principe de la méthode. La méthode proposée par Kruse et al. repose sur le **lisseur de Kalman** (algorithme de Rauch-Tung-Striebel, RTS). Contrairement au filtre de Kalman classique qui estime l'état courant uniquement à partir des mesures passées, le lisseur RTS calcule une estimation rétroactive de l'état sur une fenêtre glissante de longueur N, en exploitant à la fois les mesures passées *et* les mesures futures disponibles à l'instant courant. Cette opération de lissage produit deux types de résidus :

- les **résidus de mesure** $\hat{v}_{r,k|k}$, définis par $\hat{v}_{k-i|k} = z_{k-i} - H\hat{x}_{k-i|k}$, qui quantifient l'écart entre la mesure réelle et la mesure reconstruite par le lisseur ;

- les **résidus de processus** $\hat{w}_{q,k|k}$, définis par $\hat{w}_{k-i|k} = \hat{x}_{k-i|k} - F\hat{x}_{k-i-1|k}$, qui quantifient l'écart entre deux états lissés consécutifs.

Ces résidus, plus précis que ceux du filtre seul grâce à l'information future incorporée par le lisseur, permettent de construire des estimateurs analytiques **non biaisés** des variances du bruit de mesure et du bruit de processus :

$$\hat{\sigma}^2_{v,r} = \frac{1}{d_{v,r} \cdot L} \sum_{j=1}^{L} \sum_{i=0}^{N-1} \hat{v}^2_{r,\,j-i|j} \tag{41}$$

$$\hat{\sigma}^2_{w,q} = \frac{1}{d_{w,q} \cdot L} \sum_{j=1}^{L} \sum_{i=0}^{N-1} \hat{w}^2_{q,\,j-i|j} \tag{42}$$

où L est la longueur d'horizon sur laquelle on moyenne les résidus, et N est la taille de la fenêtre du lisseur RTS.
Les termes $d_{v,r}$ et $d_{w,q}$ sont des **facteurs de pondération analytiques** dont la propriété fondamentale est prouvée dans notre référence : ils vérifient $0 < d_{v,r} < N$ et $0 < d_{w,q} < N$, et assurent un couplage cohérent entre l'estimation de $\mathbf{Q}$ et celle de $\mathbf{R}$.
En effet, si l'on surestime $\mathbf{Q}$, le filtre attribue une plus grande partie de l'erreur d'innovation au bruit de processus, ce qui conduit mécaniquement à sous-estimer $\mathbf{R}$, et réciproquement. Les facteurs $d_{v,r}$ et $d_{w,q}$ brisent cette ambiguïté en forçant une estimation jointe cohérente, là où les méthodes classiques (*covariance matching*, IEA) divergent lorsque les deux covariances sont inconnues simultanément.

Propriété de convergence. La convergence de ces estimateurs vers les vraies valeurs des covariances est prouvée analytiquement pour les systèmes à bruit unidimensionnel, pour toute initialisation de $\mathbf{Q}_0$ et $\mathbf{R}_0$. Pour les systèmes multidimensionnels, les simulations Monte Carlo réalisées dans l'article montrent que dès $N \geq 10$, l'écart relatif entre la valeur estimée et la vraie valeur reste inférieur à 1 %, quelles que soient les conditions initiales.

Intégration d'un module d'apprentissage. La méthode AKF peut être couplée à un module de machine learning lorsque les conditions d'opération sont prévisibles à partir de grandeurs observables. Au lieu d'estimer $\mathbf{Q}$ par lissage statistique à chaque instant, un modèle supervisé apprend *hors ligne* la relation entre le contexte de fonctionnement (vitesse mesurée, dynamique, horizon temporel de sécurité) et la valeur optimale de $\mathbf{Q}$. En ligne, le modèle prédit $\mathbf{Q}_k$ à chaque instant à partir des observations courantes, et le filtre de Kalman l'utilise directement dans son étape de prédiction. Cette architecture présente deux avantages majeurs :

- elle est adaptée aux systèmes embarqués où le lisseur RTS complet serait trop coûteux en calcul, tout en conservant les bénéfices de l'adaptation dynamique de $\mathbf{Q}$
- elle permet d'intégrer des connaissances contextuelles (phase de mouvement, proximité d'obstacle) que le lisseur seul ne peut pas exploiter.

C'est précisément l'approche que nous avons implémentée dans ce TP : un modèle Random Forest entraîné sur les données réelles de l'ESP32 prédit σ_{jerk} optimal à chaque fenêtre de

mesures, ce qui reconstruit **Q** de façon adaptative avant chaque étape de prédiction du filtre de Kalman.

Applications du filtre de Kalman adaptatif

Détection de collision et véhicules autonomes. C'est l'application directe de ce TP. Dans notre implémentation, nous avons couplé un modèle Random Forest à un filtre de Kalman afin d'adapter dynamiquement la matrice **Q** en fonction du contexte de mouvement détecté. Cette architecture hybride ML-KF vise à surmonter la principale limitation du filtre de Kalman classique à paramètres fixes : son incapacité à s'adapter aux changements brusques de dynamique, tels que le passage d'une phase stationnaire à une phase d'approche rapide ou de freinage. À titre de comparaison, la littérature rapporte que des systèmes hybrides similaires permettent d'obtenir des performances supérieures à un KF seul ou à un modèle ML isolé, bien que l'évaluation quantitative de notre prototype dépasse le cadre de ce TP.

Navigation par fusion IMU/GPS. L'adaptation de **R** est particulièrement utile dans ce contexte : quand le signal GPS se dégrade en tunnel ou en zone urbaine dense, le bruit de mesure augmente brusquement. Un AKF détecte cette dégradation et augmente **R** automatiquement, sans perdre la cohérence de l'estimation de position [**?**].

Robotique industrielle. Dans le contrôle de bras robotiques ou de drones, la dynamique change selon les phases de mouvement (accélération, charge transportée, vibrations). L'AKF s'adapte naturellement à ces transitions sans reconfiguration manuelle des paramètres, ce qui est particulièrement utile en production où les conditions varient continuellement.

Estimation d'attitude de satellites. Les satellites en orbite subissent des perturbations thermiques et gravitationnelles variables selon leur position. Un AKF permet d'adapter **Q** lors des transitions ombre/soleil qui provoquent des déformations mécaniques imprévisibles, améliorant la précision du pointage.

Fusion multi-capteurs en médecine. Dans les systèmes de suivi de mouvement corporel (rééducation, chirurgie assistée), les bruits de mesure des capteurs inertiels varient selon la posture et l'activité. Un AKF adapte **R** en temps réel selon le contexte physiologique, réduisant les erreurs d'estimation sans intervention manuelle du praticien.

Référence

- T. Kruse, T. Griebel, K. Graichen, *Adaptive Kalman Filtering: Measurement and Process Noise Covariance Estimation Using Kalman Smoothing*, IEEE Access, vol. 13, pp. 11863–11875, 2025.

Implémentation : prototype embarqué pour la détection de collision

Le filtre de Kalman adaptatif reste un concept théorique tant qu'il n'est pas confronté à des données réelles issues de capteurs physiques. Pour valider les principes présentés dans la partie précédente, nous avons construit un prototype embarqué qui simule, à petite échelle, le comportement d'un système de détection de collision tel qu'on le trouve dans les véhicules autonomes.

Le choix de cette application n'est pas arbitraire. Un véhicule en mouvement illustre précisément le problème que pose un $\mathbf{Q}$ fixe : le régime dynamique change constamment entre phase stationnaire, approche d'un obstacle, freinage et risque de collision. Ces transitions sont difficiles à modéliser par un paramètre unique, ce qui en fait un cas d'usage naturel pour l'adaptation automatique de $\mathbf{Q}$.

Le prototype est constitué d'un microcontrôleur ESP32-S3 connecté à un accéléromètre MPU6050 et à un capteur ultrason HC-SR04. Il n'a pas vocation à reproduire fidèlement un véhicule réel, mais à générer des données dynamiques suffisamment riches pour tester le filtre dans des conditions variées. Les mesures brutes sont transmises par liaison série UART à MATLAB, qui exécute en temps réel le filtre de Kalman, l'adaptation de $\mathbf{Q}$ par forêt aléatoire et la classification des états de conduite.

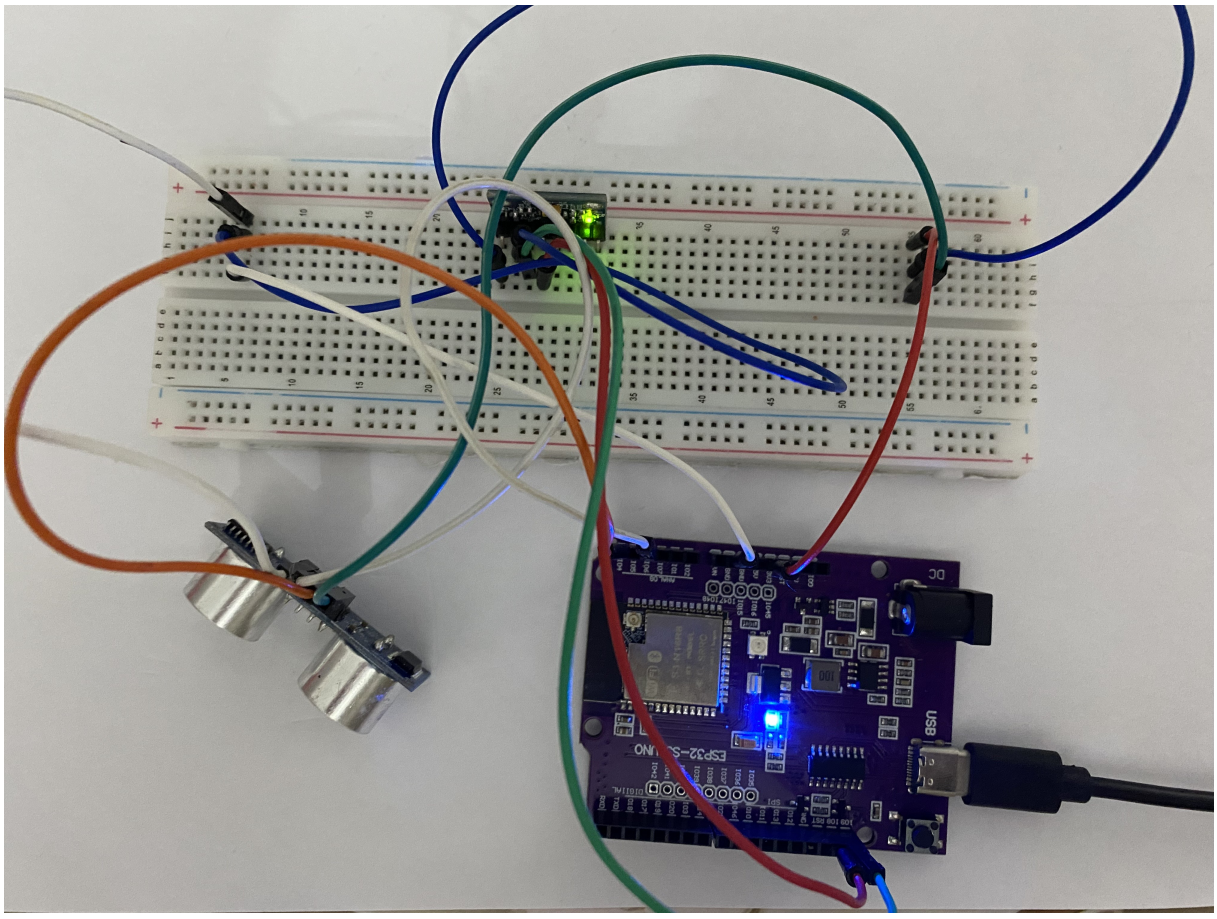


Figure 14: Figure : photographie du prototype — ESP32-S3, MPU6050, HC-SR04 sur breadboard

Câblage des capteurs. Le HC-SR04 est connecté sur TRIG = GPIO4 et ECHO = GPIO6. Le MPU6050 communique en I2C sur SDA = GPIO8 et SCL = GPIO9, avec un bus cadencé à 400 kHz. L'accéléromètre est configuré en plage $\pm 2g$ (sensibilité 16384 LSB/g) avec un filtre passe-bas interne à 20 Hz (registre DLPF). Une calibration de 200 échantillons au repos est effectuée au démarrage pour estimer le biais statique de l'axe X.

L'ESP32 envoie chaque trame au format texte `distance_m,acceleration_ms2\n`. Les lignes de débogage commençant par `#` sont ignorées par MATLAB.

[Figure : schéma bloc du système — ESP32 → UART → MATLAB (Kalman + RF)]

Modèle d'état. Le vecteur d'état à trois composantes modélise la cinématique à accélération constante (modèle CA) :

$$\mathbf{x}_k = \begin{bmatrix} d_k \\ v_k \\ a_k \end{bmatrix} \tag{43}$$

où d est la distance à l'obstacle (m), v la vitesse relative (m/s) et a l'accélération (m/s^2).

La matrice de transition d'état est :

$$A = \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad \Delta t = 0.05 \text{ s} \tag{44}$$

La matrice d'observation extrait directement la distance et l'accélération du vecteur d'état :

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{y}_k = \begin{bmatrix} d_m \\ a_m \end{bmatrix} \tag{45}$$

La matrice de bruit de mesure est construite à partir des caractéristiques des capteurs :

$$R = \begin{bmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_a^2 \end{bmatrix} = \begin{bmatrix} (0.015)^2 & 0 \\ 0 & (0.035)^2 \end{bmatrix} \tag{46}$$

avec $\sigma_d = 0.015$ m pour le HC-SR04 et $\sigma_a = 0.035$ m/s^2 pour le MPU6050.

Le bruit de processus est modélisé par un jerk (dérivée de l'accélération) de variance σ_j^2. Le vecteur d'entrée du bruit est :

$$\Gamma = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \\ 1 \end{bmatrix}, \quad Q = \sigma_j^2 \cdot \Gamma\,\Gamma^\top \tag{47}$$

Équations du filtre de Kalman. Les étapes de prédiction et de correction sont les suivantes :

$$\hat{\mathbf{x}}_{k|k-1} = A\,\hat{\mathbf{x}}_{k-1|k-1} \tag{48}$$

$$P_{k|k-1} = A\,P_{k-1|k-1}\,A^\top + Q \tag{49}$$

$$\mathbf{e}_k = \mathbf{y}_k - H\,\hat{\mathbf{x}}_{k|k-1} \tag{50}$$

$$S_k = H\,P_{k|k-1}\,H^\top + R \tag{51}$$

$$K_k = P_{k|k-1}\,H^\top\,S_k^{-1} \tag{52}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k\,\mathbf{e}_k \tag{53}$$

$$P_{k|k} = (I - K_k H)\,P_{k|k-1}\,(I - K_k H)^\top + K_k\,R\,K_k^\top \tag{54}$$

La forme de Joseph (éq. 12) est utilisée pour la mise à jour de la covariance afin de garantir la symétrie et la positivité de P.

Filtre non adaptatif (Q fixe). Dans le fichier `KF_non_adaptatif.m`, la valeur de σ_j est fixée à 1.0 m/s^3 pour toute la durée de l'acquisition. Ce choix constitue un compromis moyen : il lisse correctement les phases stationnaires mais réagit trop lentement lors des changements brusques (freinage, approche rapide). Inversement, une valeur élevée de σ_j suivrait mieux les transitions mais introduirait du bruit en phase calme.

Ce compromis statique est la principale limitation du filtre classique. Il ne peut pas s'adapter automatiquement au régime de conduite courant.

[Figure : courbes du filtre non adaptatif — distance brute vs filtrée, vitesse, accélération, TTC]

Filtre adaptatif (Q variable par Random Forest). Le fichier `KF_adaptatif.m` résout ce problème en ajustant dynamiquement σ_j à l'aide d'un modèle Random Forest de régression.

Le modèle RF est entraîné hors ligne dans `RF_adaptatif_final.m` sur un dataset de 1600 fenêtres glissantes de 20 échantillons chacune. Pour chaque fenêtre, 7 features statistiques sont calculées :

$$\mathbf{f} = \left[\, \bar{d},\ \sigma_d,\ \overline{|a|},\ \sigma_a,\ \max|\Delta d|,\ \bar{v},\ \overline{\mathrm{TTC}} \,\right] \tag{55}$$

La cible est le σ_j optimal parmi quatre niveaux discrets : $\{0.1,\ 1.0,\ 2.5,\ 5.0\}$ m/s^3. Ces niveaux correspondent respectivement aux régimes stationnaire, mouvement lent, approche active et collision imminente.

Le modèle utilise 200 arbres de décision (méthode Bagging, `fitrensemble` avec `templateTree`, profondeur max 30 splits, taille minimale de feuille 3). Le split train/test est 75%/25% avec graine fixe (`rng(42)`). Les performances obtenues sont :

- RMSE test : 0.306 m/s^3
- R^2 test : 0.974

En temps réel, le RF prédit σ_j toutes les 10 trames (soit 500 ms). La valeur prédite est lissée par un filtre exponentiel pour éviter les sauts brusques de Q :

$$\sigma_j^{(k)} = \alpha \cdot \sigma_j^{\mathrm{RF}} + (1-\alpha) \cdot \sigma_j^{(k-1)}, \quad \alpha = 0.3 \tag{56}$$

La valeur lissée est bornée dans l'intervalle $[0.1,\ 5.5]$ m/s^3, puis Q est reconstruite :

$$Q^{(k)} = \left(\sigma_j^{(k)}\right)^2 \cdot \Gamma\,\Gamma^\top \tag{57}$$

Cette mise à jour est effectuée *avant* l'étape de prédiction du filtre de Kalman à chaque cycle.

[Figure : courbes du filtre adaptatif — distance, vitesse, accélération, TTC, σ_j adaptatif]

Modèle RF de classification des états de conduite. Un second modèle Random Forest, indépendant du premier, est utilisé pour classifier l'état de conduite en temps réel. Il est entraîné en Python (`TRAIN.py`) avec `scikit-learn` sur un dataset de classification contenant les états estimés par le filtre de Kalman.

Les 5 features d'entrée sont les grandeurs estimées à chaque instant :

$$\mathbf{f}_{\text{classif}} = \left[\, d_{\text{est}},\ v_{\text{est}},\ a_{\text{est}},\ v_{\text{used}},\ \text{TTC} \,\right] \tag{58}$$

Les classes prédites sont : STATIONNAIRE, APPROCHE, FREINAGE, COLLISION RISK, ÉLOIGNEMENT. Le modèle utilise 200 arbres, avec pondération `class_weight='balanced'` pour gérer le déséquilibre des classes. La validation croisée à 5 folds donne une accuracy stable.

Le modèle Python est exporté en format `.mat` via un script dédié (`export_to_matlab.py`) qui sérialise la structure interne de chaque arbre (seuils, features, enfants, valeurs des feuilles). MATLAB parcourt ces arbres manuellement dans la fonction `rf_predict_mat` pour effectuer la classification sans dépendance externe.

La prédiction RF est exécutée tous les 2 échantillons (100 ms) et affichée dans une boîte de texte avec code couleur (vert = approche, rouge = collision, jaune = freinage, bleu = éloignement, gris = stationnaire).

[Figure : matrice de confusion et importance des features du RF classification]

Rôles distincts des deux modèles RF. Les deux modèles Random Forest ont des objectifs complémentaires. Le RF de régression agit sur le filtre lui-même : il ajuste Q pour que le filtre de Kalman réagisse plus ou moins vite selon le contexte. Le RF de classification agit en aval : il interprète les sorties du filtre pour donner une décision exploitable (état de conduite, alerte collision). Le premier améliore la qualité de l'estimation ; le second transforme cette estimation en information de haut niveau.

Comparaison des performances. En phase stationnaire, les deux filtres (adaptatif et non adaptatif) donnent des résultats similaires car $\sigma_j = 1.0$ est proche de la valeur fixe. La différence apparaît lors des transitions dynamiques :

- En phase de freinage ou d'approche rapide, le filtre adaptatif augmente σ_j (vers 2.5 ou 5.0), ce qui permet au filtre de suivre les variations rapides de distance et d'accélération sans retard.
- En phase calme, le filtre adaptatif réduit σ_j (vers 0.1), ce qui produit un lissage plus fort et réduit le bruit résiduel.
- Le TTC estimé par le filtre adaptatif est plus fiable en situation de danger car la vitesse estimée réagit plus rapidement aux changements.

Le suivi de σ_j dans le temps (5e courbe de l'affichage adaptatif) montre que le RF identifie correctement les phases de mouvement et ajuste le paramètre en conséquence.

[Figure : comparaison filtre adaptatif vs non adaptatif — superposition des courbes de distance filtrée lors d'un scénario de freinage]

[Figure : évolution de σ_j adaptatif et courbe du RF régression — prédiction vs réel sur le test set, scatter plot, convergence OOB]

Références

1. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems", *Journal of Basic Engineering*, vol. 82, pp. 35–45, 1960.
2. L. Breiman, "Random Forests", *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
3. InvenSense, "MPU-6000 and MPU-6050 Product Specification Revision 3.4", Document Number PS-MPU-6000A-00, 2013.
4. Elecfreaks, "HC-SR04 Ultrasonic Ranging Module", Datasheet, 2016.
5. Y. Bar-Shalom, X. R. Li, T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*, John Wiley & Sons, 2001.
6. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
7. MathWorks, "fitrensemble — Fit ensemble of learners for regression", Documentation MATLAB R2024a.